

# Revisiting transportation problems under Monge costs with applications to the Discrete Ordered Median Problem

Stefan Nickel <sup>1</sup>, Justo Puerto <sup>2,3</sup>, Simon Ramoser <sup>\*1</sup>, Alberto Torrejon <sup>2,3</sup>

<sup>1</sup>*Institute for Operations Research, Karlsruhe Institute of Technology, Karlsruhe, Germany*

<sup>2</sup>*Department of Statistics and Operational Research, University of Seville, Seville, Spain*

<sup>3</sup>*Institute of Mathematics, University of Seville, Seville, Spain*

## Abstract

We investigate the classical Hitchcock-Koopmans Transportation Problem under the assumption that the cost matrix satisfies the Monge property. In this setting, we derive compact formulas for optimal dual solutions based on the northwest-corner rule. As an application illustrating how these formulas yield structural insight while enhancing computational performance, we consider a broad class of facility location problems. In particular, the expressions are used within a Benders decomposition framework to derive novel mixed-integer linear programming formulations for the Discrete Ordered Median Problem with non-increasing weights. Numerical experiments validate that the resulting formulations achieve state-of-the-art performance and exhibit strong robustness across a wide range of instances.

**Keywords:** Combinatorial optimization, Monge costs, Discrete Ordered Median Problem, Benders decomposition

## 1 Introduction

The task of finding a cost-minimal transportation plan for carrying a commodity from a set of sources with given supply quantities to a set of destinations with given demands – commonly referred to as the Hitchcock-Koopmans Transportation Problem (TP) – is a classical and well-studied problem in the field of Operations Research. It has numerous applications beyond the optimal distribution of goods, such as production scheduling, supply chain management, or emergency logistics, just to name a few (see, e.g., Zhang & Xie, 2025).

An interesting special case of the TP arises when its cost matrix satisfies the *Monge property*, a notion closely connected to submodular functions (see Queyranne et al., 1998). Formally, a  $p \times q$  matrix  $c$  is a Monge matrix if  $c_{ij} + c_{i'j'} \leq c_{ij'} + c_{i'j}$  holds for all  $1 \leq i < i' \leq p$  and  $1 \leq j < j' \leq q$ . Intuitively, this inequality allows any two crossing transportation paths to be replaced by non-crossing ones without increasing the total cost, thereby ensuring the existence of an optimal transportation plan where no two paths cross. Under this assumption, the TP can be solved by a greedy algorithm in linear time. The Monge property and related structural conditions have also been exploited in other optimization problems, enabling solution algorithms that are unavailable for arbitrary cost matrices. For instance, Monge-like structures have been

---

\*Corresponding author: simon.ramoser@kit.edu

used to derive polynomial-time algorithms for problems that are NP-hard in general, including the Traveling Salesman Problem and the Quadratic Assignment Problem (see Burkard et al., 1996; Çela et al., 2018). In the context of the TP, Monge costs emerge, among others, in geometric and temporal settings where sources and destinations admit a natural sequential ordering. For example, consider  $p$  sources and  $q$  destinations indexed by integers  $i = 1, \dots, p$  and  $j = 1, \dots, q$ . If the pairwise costs are defined by the one-dimensional Euclidean distance  $c_{ij} := |i - j|$ , the resulting matrix  $c$  satisfies the Monge property. A prominent application of this is found in computer vision for measuring image similarity by comparing grayscale histograms (Werman et al., 1985; Rubner et al., 1998). Another geometric setup that induces Monge costs occurs when sources are distributed along a straight line and destinations are distributed along a parallel line. If costs again correspond to Euclidean distances between sorted source and destination nodes, the resulting cost matrix is Monge. There are, for instance, power line connection problems which can be modeled as a TP with this cost structure (Aggarwal et al., 1995). In temporal settings, the natural sequential ordering is dictated by time. An example is the job sequencing problem described by Hoffman (1963), where the goal is to minimize the maximum tardiness of all jobs. Formulating this as a TP that assigns processing time units to jobs ordered by their due dates gives rise to a Monge cost matrix. More recent applications of the TP in conjunction with Monge cost structures can be found in inventory allocation, where suitable assumptions related to product substitution costs allow for optimal greedy distribution of inventory (Tseytlin & Heese, 2019, 2023).

Despite being useful as a standalone model, in many modern applications the TP appears as a subproblem that must be solved repeatedly and efficiently. This occurs, for instance, within Benders decompositions (e.g., Fischetti et al., 2016), bilevel formulations (e.g., Hillbrecht, 2025; Singh et al., 2008), or Farkas-based valid inequality schemes (e.g., Espejo et al., 2023). In such settings, Monge costs offer an advantage that goes beyond the acceleration of solution methods: they can reveal structural insights about the overarching models and thereby enable a more efficient use of advanced optimization techniques. Our approach is to derive compact formulas for optimal dual variables of the TP under Monge costs that can be fruitfully used to gain such insights, specifically to derive novel explicit problem reformulations.

To make our ideas concrete, we conduct a case study in the field of location analysis, where many models involve a step that sorts costs before weighting them, for example, in optimal subtree location on trees (e.g., Puerto & Tamir, 2005; Pozo et al., 2024) and in hub location (e.g., Puerto et al., 2011; Pozo et al., 2021). The sorting step imposes a sequential structure on the allocation costs, making these models a natural place in which Monge matrices can appear. Particularly, we consider the Discrete Ordered Median Problem (DOMP), which generalizes a large variety of location problems. Given a number  $n$  of potential facility locations and a number  $p \leq n$  of facilities that are to be opened, the goal is to choose the  $p$  best facility locations in the sense that weighted delivery costs from these locations to customer locations are minimized. The weights are order-dependent, meaning that they are applied to the non-decreasingly sorted delivery costs. By appropriate choices of the weight vector, the DOMP encompasses many classical location models as special cases, including the  $p$ -median,  $p$ -center, and  $k$ -centrum problems. In this study, we consider instances with non-increasing order-dependent weights. Such objectives occur, for instance, in obnoxious facility problems, where “undesired” facilities, such as wind turbines, power plants or landfills, must be located (Church & Drezner, 2022). Motivated by recent advances in this area (e.g., Bigler, 2024; Kalczynski & Drezner, 2026), we apply Benders decomposition to a mixed-integer linear programming (MILP) formulation for the DOMP proposed by Labbé et al. (2017). The Benders subproblem for generating optimality cuts is a TP whose cost matrix becomes Monge under the assumption of non-increasing weights. Because these cuts are derived from optimal dual solutions to this TP, our formulas yield explicit representations of the Benders cuts, leading to

new models.

## 1.1 Literature review

### 1.1.1 The Transportation Problem

Although the history of the TP dates back to the 18th century when Gaspard Monge was concerned with transporting earth (Monge, 1781), the problem’s greatest developments occurred in the 20th century. While Kantorovich (1942) generalized the problem, thereby grounding the theory of optimal transport, Hitchcock (1941) and Koopmans (1949) independently formalized the discrete version considered in this paper. Early major contributions to the latter include the application of the simplex method by Dantzig (1951) and the network flow algorithm developed by Ford & Fulkerson (1956). Since then, researchers from a wide range of disciplines have worked on the TP, resulting in a vast body of literature. We therefore limit this brief historical overview to the above mentioned milestones and refer to Zhang & Xie (2025) for a recent review of the problem with modern perspectives. A broad overview of extensions of the TP, together with their corresponding developments and mathematical formulations, is presented by Kacher & Singh (2021). Structural insights and polyhedral foundations related to the TP are treated in the classical textbook by Yemelichev et al. (1984). Finally, for optimization problems under Monge cost structures, the focus of this paper, the standard references remain the extensive surveys by Burkard et al. (1996) and Burkard (2007), which include a discussion of the TP. In this context, a seminal work is due to Hoffman (1963), who showed that the TP can be solved greedily to optimality when the costs satisfy a certain structural condition. Since the underlying idea of this condition was already contained in Monge’s 1781 work, Hoffman named this property after Monge.

### 1.1.2 The Discrete Ordered Median Problem

At the beginning of the century, Nickel (2001) laid the foundations of the DOMP literature by formulating the problem as a mixed-integer quadratic program and subsequently linearizing it to obtain the first MILP model for the DOMP. The early developments on the problem are comprehensively summarized in the textbook by Nickel & Puerto (2005), including improved models as well as exact and heuristic solution algorithms by Boland et al. (2006); Domínguez-Marín (2003); and Domínguez-Marín et al. (2005). In 2009, Marín et al. (2009) proposed a DOMP model capable of solving specific instances with up to  $n = 100$  potential facility locations within reasonable time. A new modeling approach based on rewriting the ordered median objective function of the DOMP in terms of  $k$ -sums and exploiting known theory from  $k$ -sum optimization (Nickel & Velten, 2017; Puerto et al., 2017) gave rise to further improvements. Particularly, this idea has led to DOMP formulations that are capable of solving a wide range of instances with up to  $n = 200$  locations (Marín et al., 2020). More advanced decomposition-based optimization techniques such as Branch-and-Price (Deleplanque et al., 2020), Benders decomposition (Ljubić et al., 2024), or customized Branch-and-Bound with special DOMP subproblems (Cherkesly et al., 2025) were applied successfully, resulting in state-of-the-art model performance.

It is well-known that the DOMP contains obnoxious facility location problems as special cases, including the obnoxious  $p$ -median and  $p$ -center problems. Within the DOMP literature, such instances are recognized as particularly challenging, largely because negative objective weights substantially affect the problem structure. Consequently, recent computational studies employ obnoxious criteria both to demonstrate the strength of their formulations and to find their computational limits (Ljubić et al., 2024; Cherkesly et al., 2025). The

obnoxious  $p$ -median and  $p$ -center problems have also been studied independently of the DOMP. In recent years, new models and solution approaches have been proposed (see, e.g., Chiang & Lin, 2017; Colmenar et al., 2016; Herrán et al., 2020; Bigler, 2024; Kalczynski & Drezner, 2026). For a broader overview of obnoxious facility location problems, we refer to the survey by Church & Drezner (2022).

## 1.2 Contributions

With this work, we contribute to the literature as follows.

- We explicitly derive compact expressions for optimal dual variables of TPs under Monge costs through a close analysis of the primal solution generated by the greedy northwest-corner algorithm.
- We show how the results on TPs under Monge costs can be exploited when such problems appear as substructures within advanced optimization frameworks by deriving novel MILP formulations for the DOMP with non-increasing objective weights.
- We conduct an extensive computational study comparing the proposed formulations with state-of-the-art DOMP models on a broad range of instances, including prominent obnoxious facility location problems, and demonstrate that the proposed models achieve highly competitive, and in several cases superior, computational performance.

The remainder of this paper is structured as follows. Section 2 starts with a detailed presentation of the TP under Monge costs, and recalls the greedy northwest-corner algorithm together with its extension for retrieving a dual solution. Building on this, it is shown how a so-obtained dual solution can be expressed compactly using formulas that encode the primal solution generated by the northwest-corner algorithm. In Section 3, the DOMP is introduced formally along with a MILP model from the literature to which a Benders reformulation is applied. The dual solutions identified in Section 2 give rise to two new exponential MILP formulations for DOMP with non-increasing weights; we conclude the section with several theoretical and practical remarks. Section 4 reports computational experiments that compare the new formulations with two state-of-the-art DOMP models. Section 5 ends the paper with a concise summary and suggestions for further research.

## 2 The Transportation Problem under Monge costs

This section develops our core results regarding the TP under the assumption of Monge costs. In Subsection 2.1, we define the TP under Monge costs and recall the well-known northwest-corner method that finds optimal primal solutions to the studied problem. In addition, we prove a characterization of these solutions that is solely based on the given supply and demand quantities (Proposition 2.2). In Subsection 2.2, we proceed to our main results of this section: compact expressions for solutions to the dual TP under Monge costs (Theorems 2.5 and 2.6). This is achieved by analyzing a known extension of the northwest-corner algorithm that recovers an optimal dual solution encoding the corresponding primal solution. To our knowledge, these explicit compact expressions have not appeared in the literature before.

### 2.1 Problem definition and the northwest-corner solution method

Let  $p \in \mathbb{Z}_{\geq 1}$  be the number of supply nodes with supplies  $s \in \mathbb{R}_{\geq 0}^p$ , and  $q \in \mathbb{Z}_{\geq 1}$  be the number of demand nodes with demands  $d \in \mathbb{R}_{\geq 0}^q$ . Moreover, let  $c \in \mathbb{R}_{\geq 0}^{p \times q}$  be the transportation cost matrix, where entry  $c_{ij}$

prescribes the costs of carrying one unit of the transported commodity from supply node  $i$  to demand node  $j$ . The goal is to find the non-negative quantities that should be transferred between any pair of supply and demand nodes such that all supplies and demands are exactly satisfied and the total transportation cost is minimal. It is easy to see that this is possible if and only if supplies and demands are “balanced”, so we assume  $\sum_{i=1}^p s_i = \sum_{j=1}^q d_j$ .

Consider the standard LP formulation of the classical (balanced) TP, first given by Hitchcock (1941), and stated here as model (1). It employs continuous, non-negative decision variables  $x_{ij}$  for all  $(i, j) \in [p] \times [q]$ <sup>1</sup>, representing the quantity shipped from supply node  $i$  to demand node  $j$ .

$$\text{TP : min } \sum_{i=1}^p \sum_{j=1}^q c_{ij} x_{ij} \quad (1a)$$

$$\text{s.t. } \sum_{j=1}^q x_{ij} = s_i \quad \forall i \in [p] \quad (1b)$$

$$\sum_{i=1}^p x_{ij} = d_j \quad \forall j \in [q] \quad (1c)$$

$$x_{ij} \geq 0 \quad \forall i \in [p], j \in [q] \quad (1d)$$

Feasibility of this LP is guaranteed by the balancedness condition, and the problem is clearly bounded. Moreover, since the constraint matrix is totally unimodular, any basic feasible solution will be integer-valued if the right-hand sides are integral (see, e.g., Bazaraa et al., 2010).

Throughout this section, we assume that  $c$  satisfies the Monge property. Recall from the introduction that this requires the inequality  $c_{ij} + c_{i'j'} \leq c_{ij'} + c_{i'j}$  to hold for all  $1 \leq i < i' \leq p$ ,  $1 \leq j < j' \leq q$ . In that case, the greedy northwest-corner rule always yields an optimal solution to (1). This algorithm starts in the northwest-corner of the  $p \times q$  variable matrix, that is, at index pair  $(i, j) = (1, 1)$  and sets  $x_{ij}$  to the largest number of units possible to be transported from supplier  $i$  to demand point  $j$ . After updating the supplies and demands according to the already sent units, the procedure advances to cell  $(i + 1, j)$  if supplier  $i$  has reached its capacity or to cell  $(i, j + 1)$  if demand  $j$  is satisfied. Like this, the procedure iterates until all available units are shipped. Algorithm 1 is the version of this greedy northwest-corner rule presented by Queyranne et al. (1998) and specialized to the classical TP.

Algorithm 1 moves along a staircase-shaped path from cell  $(1, 1)$  to cell  $(p, q)$ : between cell  $a^t$  and  $a^{t+1}$ , it either advances one row (“down”) or one column (“right”) on the  $p \times q$  grid. This movement is well understood and can be characterized as follows.

**Observation 2.1.** *We have  $(i, j) \in \{a^1, \dots, a^T\} =: \mathcal{A}$  if and only if one of the following holds:*

$$(i) \ (i, j) = (1, 1)$$

$$(ii) \ (i - 1, j) \in \mathcal{A} \text{ and } \sum_{k=1}^{i-1} s_k \leq \sum_{k=1}^j d_k \text{ and } i - 1 < p$$

$$(iii) \ (i, j - 1) \in \mathcal{A} \text{ and } (\sum_{k=1}^i s_k > \sum_{k=1}^{j-1} d_k \text{ or } i = p)$$

*Proof.* The algorithm is initialized with  $a^1 = (1, 1)$ . From there, it proceeds by moving one row downwards or one column to the right. (ii) characterizes a downward movement from  $(i - 1, j)$  to  $(i, j)$ . This happens exactly when  $i - 1 < p$  and the total supply up to  $i - 1$  does not exceed the total demand up to  $j$ , that is,

---

<sup>1</sup>For  $n \in \mathbb{Z}_{\geq 1}$ , we use the notation  $[n] = \{1, \dots, n\}$ .

---

**Algorithm 1:** Northwest-corner rule

---

```
1  $x \leftarrow \{\}$ 
2  $\eta \leftarrow \{\}$ 
3  $\delta_i^1 \leftarrow s_i \quad \forall i \in [p]$  // remaining supplies
4  $\delta_j^2 \leftarrow d_j \quad \forall j \in [q]$  // remaining demands
5  $a \leftarrow (1, 1)$  // start in the northwest-corner
6  $t \leftarrow 0$ 
7 while  $a_2 \leq q$  do
8    $t \leftarrow t + 1$ 
9    $a^t \leftarrow a$ 
10   $x_{a^t} \leftarrow \min\{\delta_{a_1^t}^1, \delta_{a_2^t}^2\}$ ; append  $(a^t, x_{a^t})$  to  $x$ 
11   $\delta_{a_1^t}^1 \leftarrow \delta_{a_1^t}^1 - x_{a^t}$ 
12   $\delta_{a_2^t}^2 \leftarrow \delta_{a_2^t}^2 - x_{a^t}$ 
13  if  $\delta_{a_1^t}^1 = 0$  and  $a_1^t < p$  then
14     $\eta_t \leftarrow 1$ ; append  $\eta_t$  to  $\eta$ 
15     $a_1^t \leftarrow a_1^t + 1$  // move down
16  else
17     $\eta_t \leftarrow 2$ ; append  $\eta_t$  to  $\eta$ 
18     $a_2^t \leftarrow a_2^t + 1$  // move right
19   $a \leftarrow a^t$ 
20  $T \leftarrow t$ 
21 return  $\{(a^1, x_{a^1}), \dots, (a^T, x_{a^T})\}, \{\eta_1, \dots, \eta_{T-1}\}$ 
```

---

when  $\sum_{k=1}^{i-1} s_k \leq \sum_{k=1}^j d_k$ . Similarly, (iii) describes a move to the right from  $(i, j-1)$  to  $(i, j)$ . Such a move occurs exactly when the total supply up to  $i$  exceeds the total demand up to  $j$ , that is,  $\sum_{k=1}^i s_k > \sum_{k=1}^{j-1} d_k$ , or when  $i = p$ . Note that in the latter case, it holds  $\sum_{k=1}^i s_k = \sum_{k=1}^{j-1} d_k$ . Overall, we see that (i), (ii), (iii) inductively describe the cells  $a^1, \dots, a^T$  traversed by Algorithm 1.  $\square$

Staircase patterns in transportation matrices have been studied from a broader perspective by Dahl (2008). The pattern of a transportation matrix  $(x_{ij})_{\substack{i \in [p] \\ j \in [q]}}$  refers to the positions of its nonzero entries. In his work, Dahl characterizes transportation matrices with a given staircase pattern in terms of certain “critical positions” within that pattern. In particular, he shows that if specific inequalities involving the input vectors  $s$  and  $d$  hold for each of these critical positions, then the path generated by the northwest-corner rule is contained in that pattern. Since we focus exclusively on the staircase pattern traced by the northwest-corner algorithm, we can provide, for our purposes, a more convenient characterization that does not rely on critical positions. This novel result is stated in Proposition 2.2.

**Proposition 2.2.** *For the cells  $\mathcal{A} = \{a^1, \dots, a^T\}$  traversed by Algorithm 1, it holds*

$$\mathcal{A} = \left\{ (i, j) \in [p] \times [q] \mid \left( \sum_{k=1}^{i-1} s_k \leq \sum_{k=1}^j d_k \vee i = 1 \right) \wedge \left( \sum_{k=1}^i s_k > \sum_{k=1}^{j-1} d_k \vee i = p \vee j = 1 \right) \right\}. \quad (2)$$

*Proof.* The key structural insight leading to this claim is that if a tuple  $(i, j)$  lies strictly “below” the staircase path  $\mathcal{A}$ ,  $\sum_{k=1}^{i-1} s_k \leq \sum_{k=1}^j d_k$  cannot hold true. Similarly, if  $(i, j)$  lies strictly “above” the staircase,  $\sum_{k=1}^i s_k > \sum_{k=1}^{j-1} d_k$  cannot hold true. Intuitively, this means that the condition in Observation 2.1 (ii) for down moves of Algorithm 1 can never hold below the staircase while the condition in Observation 2.1 (iii) for right moves

cannot hold above the staircase. In that sense, the cells  $\mathcal{A}$  are exactly at the intersection between feasible down moves and right moves of Algorithm 1.

We start by proving the aforementioned structural insight. Let  $(i, j) \in [p] \times [q]$  such that  $i > a_1^t$  for every  $t \in [T]$  with  $a_2^t = j$ , that is,  $(i, j) \notin \mathcal{A}$  lies strictly below the staircase. Suppose for contradiction that  $\sum_{k=1}^{i-1} s_k \leq \sum_{k=1}^j d_k$ . The staircase-like movement of Algorithm 1 implies the existence of  $\tilde{i} < i$  such that  $(\tilde{i}, j) \in \mathcal{A}$  and  $(\tilde{i}, j+1) \in \mathcal{A}$ . Note that it must hold  $j < q$  because otherwise  $(i, j) \in \mathcal{A}$ . By Observation 2.1 (iii), we have  $\sum_{k=1}^{\tilde{i}} s_k > \sum_{k=1}^j d_k$  or  $\tilde{i} = p$ . The latter is not possible since  $\tilde{i} < i \leq p$ , so

$$\sum_{k=1}^j d_k < \sum_{k=1}^{\tilde{i}} s_k \leq \sum_{k=1}^{i-1} s_k \leq \sum_{k=1}^j d_k.$$

We conclude from this contradiction that the assumption was wrong.

Similarly, let  $(i, j) \in [p] \times [q]$  such that  $j > a_2^t$  for every  $t \in [T]$  with  $a_1^t = i$ , that is,  $(i, j) \notin \mathcal{A}$  lies strictly above the staircase. Suppose for contradiction that  $\sum_{k=1}^i s_k > \sum_{k=1}^{j-1} d_k$  or  $i = p$ . The latter is not possible since  $(p, q) \in \mathcal{A}$  and thus, the assumption on  $j$  would imply  $j > q$ . So, assume  $i < p$  and  $\sum_{k=1}^i s_k > \sum_{k=1}^{j-1} d_k$ . The staircase-like movement of Algorithm 1 implies the existence of  $\tilde{j} < j$  such that  $(i, \tilde{j}) \in \mathcal{A}$  and  $(i+1, \tilde{j}) \in \mathcal{A}$ . By Observation 2.1 (ii), we have  $\sum_{k=1}^i s_k \leq \sum_{k=1}^{\tilde{j}} d_k$ . Therefore,

$$\sum_{k=1}^{j-1} d_k < \sum_{k=1}^i s_k \leq \sum_{k=1}^{\tilde{j}} d_k \leq \sum_{k=1}^{j-1} d_k.$$

This shows the second part of the key structural insight.

For the remainder of this proof, the right-hand side of (2) is denoted by  $\mathcal{B}$ . We proceed by proving  $\mathcal{A} \subseteq \mathcal{B}$ . For this, let  $(i, j) \in \mathcal{A}$ . By Observation 2.1 three scenarios are possible. (i) If  $(i, j) = (1, 1)$ , trivially  $(i, j) \in \mathcal{B}$ . (ii) If  $(i-1, j) \in \mathcal{A}$  and  $\sum_{k=1}^{i-1} s_k \leq \sum_{k=1}^j d_k$  and  $i-1 < p$ , we distinguish multiple cases. If  $i = p$  or  $j = 1$ , clearly  $(i, j) \in \mathcal{B}$ . So, let  $j > 1$ ,  $i < p$  and suppose for contradiction  $(i, j) \notin \mathcal{B}$ , that is,  $\sum_{k=1}^i s_k \leq \sum_{k=1}^{j-1} d_k$ . But this is not possible due to our structural property and the fact that  $(i+1, j-1) \in [p] \times [q]$  lies strictly below the staircase. Consequently,  $\sum_{k=1}^i s_k > \sum_{k=1}^{j-1} d_k$  and  $(i, j) \in \mathcal{B}$ . (iii) Now, let  $(i, j-1) \in \mathcal{A}$  and  $(\sum_{k=1}^i s_k > \sum_{k=1}^{j-1} d_k$  or  $i = p$ ). Suppose for contradiction  $(i, j) \notin \mathcal{B}$ , that is,  $i > 1$  and  $\sum_{k=1}^{i-1} s_k > \sum_{k=1}^j d_k$ . By the fact that the considered TP is balanced, this strict inequality can only hold if  $j < q$ . As before, we therefore obtain a contradiction to the structural property since  $(i-1, j+1) \in [p] \times [q]$  is strictly above the staircase.

To prove  $\mathcal{B} \subseteq \mathcal{A}$ , let  $(i, j) \in \mathcal{B}$  and suppose  $(i, j) \notin \mathcal{A}$ . Then  $(i, j)$  must either be strictly below or above the staircase path given by  $\mathcal{A}$ . If it is strictly below,  $i = 1$  cannot hold, so we must have  $\sum_{k=1}^{i-1} s_k \leq \sum_{k=1}^j d_k$  by the definition of  $\mathcal{B}$ . However, this is an immediate contradiction to the structural property. If  $(i, j)$  is strictly above the staircase, it is neither possible that  $i = p$  nor that  $j = 1$ . Hence  $\sum_{k=1}^i s_k > \sum_{k=1}^{j-1} d_k$ , again contradicting the known structural insight.

Altogether, we see that  $\mathcal{A} = \mathcal{B}$ . □

## 2.2 Compact formulas for dual solutions

Introducing dual variables  $u_i$  for constraints (1b) and  $v_j$  for (1c), one obtains the following dual LP for problem (1).

$$\max \quad \sum_{i=1}^p s_i u_i + \sum_{j=1}^q d_j v_j \quad (3a)$$

$$\text{s.t.} \quad u_i + v_j \leq c_{ij} \quad \forall i \in [p], j \in [q] \quad (3b)$$

$$u_i, v_j \in \mathbb{R} \quad \forall i \in [p], j \in [q] \quad (3c)$$

Feasibility and boundedness of the primal LP (1) imply, by strong duality, that the dual LP (3) attains a finite optimum. Queyranne et al. (1998) also provide an algorithm for solving the dual based on the output of the northwest-corner rule for the primal. This algorithm backtracks the cells  $a^1, \dots, a^T$ , thereby assigning values to the dual variables depending on whether the primal algorithm moved down or right when producing the northwest-corner solution. The precise steps are shown in Algorithm 2 which assumes the output of Algorithm 1 as input.

---

### Algorithm 2: Dual backward recursion

---

```

1 Declare  $u \in \mathbb{R}^p$  and  $v \in \mathbb{R}^q$ 
2  $v_q \leftarrow 0$ 
3  $\eta_T \leftarrow 1$  //  $u_p$  will be set in the first iteration
4 for  $t$  from  $T$  down to 1 do
5   if  $\eta_t = 1$  then
6      $u_{a_1^t} \leftarrow c_{a^t} - v_{a_2^t}$ 
7   else if  $\eta_t = 2$  then
8      $v_{a_2^t} \leftarrow c_{a^t} - u_{a_1^t}$ 
9 return  $u, v$ 
```

---

While Algorithm 2 computes dual variables by backtracking the northwest-corner path from  $(p, q)$  to  $(1, 1)$ , we show in Lemma 2.3 below that the procedure can simply be reversed without compromising its correctness.

---

### Algorithm 3: Dual forward recursion

---

```

1 Declare  $u \in \mathbb{R}^p$  and  $v \in \mathbb{R}^q$ 
2  $u_1 \leftarrow 0$ 
3  $\eta_0 \leftarrow 2$  //  $v_1$  will be set in the first iteration
4 for  $t$  from 1 to  $T$  do
5   if  $\eta_{t-1} = 1$  then
6      $u_{a_1^t} \leftarrow c_{a^t} - v_{a_2^t}$ 
7   else if  $\eta_{t-1} = 2$  then
8      $v_{a_2^t} \leftarrow c_{a^t} - u_{a_1^t}$ 
9 return  $u, v$ 
```

---

**Lemma 2.3** (Correctness of Algorithm 3). *Let  $(u, v)$  be the solution returned by Algorithm 2. Then, Al-*

gorithm 3 returns  $(u - u_1, v + u_1)$ .<sup>2</sup> This solution is feasible for (3) and has the same objective value as  $(u, v)$ .

*Proof.* We show the first part of the lemma by induction. Let  $(\tilde{u}, \tilde{v})$  be the solution returned by Algorithm 3 after  $t$  iterations. For  $t = 1$ , we have  $(\tilde{u}_1, \tilde{v}_1) = (0, c_{11})$  and thus, immediately  $\tilde{u}_1 = 0 = u_1 - u_1$ . Moreover,  $u_1 + v_1 = c_{11} = \tilde{v}_1$ , so the claim holds after iteration  $t = 1$ . For the inductive step, suppose that the claim holds true in some iteration  $t < T$ . That is,  $(\tilde{u}_{a_1^t}, \tilde{v}_{a_2^t}) = (u_{a_1^t} - u_1, v_{a_2^t} + u_1)$ . For iteration  $t + 1$ , there are two cases. If  $\eta_t = 1$ , the algorithm sets  $\tilde{u}_{a_1^{t+1}} \leftarrow c_{a_1^{t+1}} - \tilde{v}_{a_2^{t+1}}$ . Since this corresponds to a “down” movement, we have  $a^{t+1} = (a_1^t + 1, a_2^t)$  implying  $\tilde{v}_{a_2^{t+1}} = \tilde{v}_{a_2^t} = v_{a_2^t} + u_1$ . Hence,  $\tilde{u}_{a_1^{t+1}} = c_{a_1^{t+1}} - v_{a_2^t} - u_1 = u_{a_1^{t+1}} - u_1$ . So, we have  $(\tilde{u}_{a_1^{t+1}}, \tilde{v}_{a_2^{t+1}}) = (u_{a_1^{t+1}} - u_1, v_{a_2^{t+1}} + u_1)$ . Analogously, one can show the case  $\eta_t = 2$ . Now, it follows by induction that Algorithm 3 indeed returns  $(u - u_1, v + u_1)$ .

Feasibility of  $(u - u_1, v + u_1)$  is implied by the feasibility of  $(u, v)$ : for every  $i \in [p]$  and  $j \in [q]$ , it holds  $(u_i - u_1) + (v_j + u_1) = u_i + v_j \leq c_{ij}$ . Moreover, the objective values coincide:

$$\sum_{i=1}^p s_i(u_i - u_1) + \sum_{j=1}^q d_j(v_j + u_1) = \sum_{i=1}^p s_i u_i + \sum_{j=1}^q d_j v_j - u_1 \underbrace{\left( \sum_{i=1}^p s_i - \sum_{j=1}^q d_j \right)}_{=0} = \sum_{i=1}^p s_i u_i + \sum_{j=1}^q d_j v_j.$$

To conclude, the correctness of Algorithm 3 follows from the correctness of Algorithm 2.  $\square$

We remark that the initialization  $u_1 \leftarrow 0$  of Algorithm 3 is somewhat arbitrary and could be replaced by any constant and one may even initialize  $v_1$  instead of  $u_1$  as the following lemma shows.

**Lemma 2.4.** *Let  $(\bar{u}, \bar{v})$  be the solution returned by Algorithm 3 and let  $b \in \mathbb{R}$ . Then, the following holds.*

(i) *If the initialization is changed to  $u_1 \leftarrow b$ , the output is  $(\bar{u} + b, \bar{v} - b)$ .*

(ii) *If the initialization is changed to  $v_1 \leftarrow b$ ,  $\eta_0 \leftarrow 1$ , the output is  $(\bar{u} + (\bar{v}_1 - b), \bar{v} - (\bar{v}_1 - b))$ .*

*In both cases, the output remains feasible for (3) with the same objective value as  $(\bar{u}, \bar{v})$ .*

The proof of Lemma 2.4 follows the same argumentation used to prove Lemma 2.3. It is therefore omitted.

For our application in Section 3, we are particularly interested in optimal dual variables  $(u, v)$ . It will be helpful to rewrite the recursive formulas for  $u$  and  $v$  given in Algorithm 3 into a compact expression that encodes the structure of the primal solution.

For  $i \in \{2, \dots, p\}$ , define  $j_i := \min\{j \in [q] \mid (i, j) \in \{a^1, \dots, a^T\}\}$ . As the northwest-corner rule moves in a staircase path from cell  $(1, 1)$  to cell  $(p, q)$  while advancing either one row or one column per iteration, the existence of  $j_i$  is guaranteed for all  $i \in \{2, \dots, p\}$ . Observe that Algorithm 3 sets  $u_i$  in iteration  $t$  where  $a^t = (i, j_i)$ . Using this notation, the next theorem provides compact formulas for  $u$  and  $v$  computed by Algorithm 3. This is the first variant of our main result in Section 2.

**Theorem 2.5.** *Algorithm 3 returns the following solution:*<sup>3</sup>

$$u_i = \sum_{k=2}^i (c_{kj_k} - c_{k-1, j_k}) \quad i \in [p], \quad (4)$$

$$v_j = c_{1j} + \sum_{\substack{k \in [p], k \geq 2 \\ j_k \leq j}} (c_{kj} - c_{k-1, j} - c_{kj_k} + c_{k-1, j_k}) \quad j \in [q]. \quad (5)$$

<sup>2</sup>For a vector  $y \in \mathbb{R}^d$  and a scalar  $\alpha \in \mathbb{R}$ , the sum  $y + \alpha := (y_i + \alpha)_{i \in [d]}$  is understood componentwise.

<sup>3</sup>We assume the convention that an empty sum has value equal to 0.

*Proof.* First, we show (4) by induction. For  $i = 1$ , the sum is empty, so  $u_1 = 0$  which is coherent with how Algorithm 3 sets  $u_1$ . For the inductive step suppose the formula is correct for some  $i \in [p-1]$ , that is,  $u_i = \sum_{k=2}^i (c_{kj_k} - c_{k-1,j_k})$ . Recall that, at iteration  $t$  with  $a^t = (i, j_i)$ , Algorithm 3 sets  $u_i \leftarrow c_{ij_i} - v_{j_i}$ . It proceeds by setting  $v_{j'}$  for all  $j' \in \{j_i + 1, \dots, j_{i+1}\}$ . (If this set is empty, then  $j_{i+1} = j_i$  and the procedure continues directly with  $u_{i+1}$ .) Then, it assigns  $u_{i+1} \leftarrow c_{i+1,j_{i+1}} - v_{j_{i+1}}$ . But  $v_{j_{i+1}} = c_{ij_{i+1}} - u_i$ , so  $u_{i+1} = u_i + c_{i+1,j_{i+1}} - c_{ij_{i+1}} = \sum_{k=2}^{i+1} (c_{kj_k} - c_{k-1,j_k})$ . We used the induction hypothesis in the last equation. This completes the inductive proof.

To show (5), we make use of the previously shown formula (4). In particular, let  $j \in [q]$  and  $(i_j, j) \in \{a^1, \dots, a^T\}$ . Then, Algorithm 3 assigns  $v_j \leftarrow c_{i_j j} - u_{i_j}$ . So, using (4),  $v_j = c_{i_j j} - \sum_{k=2}^{i_j} (c_{kj_k} - c_{k-1,j_k})$ . For  $i_j = 1$ , it holds that  $u_{i_j} = 0$ , so the algorithm sets  $v_j \leftarrow c_{1j}$  which corresponds to the value given by the formula. For  $i_j \geq 2$ , observe that  $i_j = \max\{k \in [p] \mid k \geq 2 : j_k \leq j\}$ . Therefore, we may rewrite  $v_j = c_{i_j j} - \sum_{\substack{k \in [p], k \geq 2 \\ j_k \leq j}} (c_{kj_k} - c_{k-1,j_k})$ . Moreover, it is easy to see by a telescoping sum argument that  $c_{i_j j} = c_{1j} + \sum_{\substack{k \in [p], k \geq 2 \\ j_k \leq j}} (c_{kj} - c_{k-1,j} - c_{kj_k} + c_{k-1,j_k})$ . Putting everything together, we obtain  $v_j = c_{1j} + \sum_{\substack{k \in [p], k \geq 2 \\ j_k \leq j}} (c_{kj} - c_{k-1,j} - c_{kj_k} + c_{k-1,j_k})$ .  $\square$

With Theorem 2.5, we have a compact formula for computing a solution to (3). The formula encodes the staircase path traversed by the corresponding primal solution obtained from the northwest-corner rule. This encoding is mainly based on the indices  $j_i$  for all  $i \in \{2, \dots, p\}$ . Alternatively, one may consider an encoding based on indices  $i_j$ : for all  $j \in \{1, \dots, q-1\}$ , define  $i_j := \max\{i \in [p] \mid (i, j) \in \{a^1, \dots, a^T\}\}$ . If Algorithm 3 is initialized with  $v_1 \leftarrow 0$  and  $\eta_0 \leftarrow 1$ , it sets  $v_j$  in iteration  $t$  where  $a^t = (i_j, j)$  and we obtain another pair of compact formulas for  $u$  and  $v$ . These formulas constitute the second variant of our main result in this section.

**Theorem 2.6.** *If Algorithm 3 is initialized with  $v_1 \leftarrow 0$  and  $\eta_0 \leftarrow 1$ , it returns the following solution:*

$$u_i = c_{i1} + \sum_{\substack{k \in [q], k \geq 2 \\ i_{k-1} \leq i}} (c_{ik} - c_{i,k-1} - c_{i_{k-1}k} + c_{i_{k-1},k-1}) \quad i \in [p], \quad (6)$$

$$v_j = \sum_{k=2}^{j} (c_{i_{k-1}k} - c_{i_{k-1},k-1}) \quad j \in [q]. \quad (7)$$

Theorem 2.6 can be proved in the same fashion as Theorem 2.5. If  $(\bar{u}, \bar{v})$  is the solution given by Theorem 2.5, then, according to Lemma 2.4, the values  $(u, v)$  described by Theorem 2.6 satisfy  $(u, v) = (\bar{u} + c_{11}, \bar{v} - c_{11})$ . Moreover, if  $(\bar{u}, \bar{v})$  is an optimal solution to (3), then so is  $(u, v)$ .

We continue with a useful characterization of the indices  $j_i$ ,  $i \in \{2, \dots, p\}$  and  $i_j$ ,  $j \in \{1, \dots, q-1\}$  that will be helpful for our application of the TP.

**Corollary 2.7.** *The following holds for  $j_i$  and  $i_j$  as obtained by Algorithm 1:*

- (i) For all  $i \in \{2, \dots, p\}$ , it holds  $j_i = \min\{j \in [q] \mid \sum_{k=1}^{i-1} s_k \leq \sum_{k=1}^j d_k\}$ .
- (ii) For all  $j \in \{1, \dots, q-1\}$ , it holds  $i_j = \max\{i \in [p] \mid \sum_{k=1}^{i-1} s_k \leq \sum_{k=1}^j d_k\}$ .

*Proof.* To prove (i), let  $i \in \{2, \dots, p\}$ . By definition of  $j_i$  and by Proposition 2.2, we obtain

$$j_i = \min\{j \in [q] \mid (i, j) \in \{a^1, \dots, a^T\}\}$$

$$\begin{aligned}
&\stackrel{(2)}{=} \min \left\{ j \in [q] \mid \left( \sum_{k=1}^{i-1} s_k \leq \sum_{k=1}^j d_k \vee i = 1 \right) \wedge \left( \sum_{k=1}^i s_k > \sum_{k=1}^{j-1} d_k \vee i = p \vee j = 1 \right) \right\} \\
&= \min \left\{ j \in [q] \mid \sum_{k=1}^{i-1} s_k \leq \sum_{k=1}^j d_k \right\}.
\end{aligned}$$

For the last equation, observe that the only condition that could possibly prevent the minimum from choosing  $j = 1$  is the lower bound given by  $\sum_{k=1}^{i-1} s_k \leq \sum_{k=1}^j d_k$ .

To prove (ii), let  $j \in \{1, \dots, q-1\}$ . By definition of  $i_j$  and by Proposition 2.2, we obtain

$$\begin{aligned}
i_j &= \max \{ i \in [p] \mid (i, j) \in \{a^1, \dots, a^T\} \} \\
&\stackrel{(2)}{=} \max \left\{ i \in [p] \mid \left( \sum_{k=1}^{i-1} s_k \leq \sum_{k=1}^j d_k \vee i = 1 \right) \wedge \left( \sum_{k=1}^i s_k > \sum_{k=1}^{j-1} d_k \vee i = p \vee j = 1 \right) \right\} \\
&= \max \left\{ i \in [p] \mid \sum_{k=1}^{i-1} s_k \leq \sum_{k=1}^j d_k \right\}.
\end{aligned}$$

Similar as before, the only condition preventing the maximum from choosing  $i = p$  is  $\sum_{k=1}^{i-1} s_k \leq \sum_{k=1}^j d_k$ , which explains the last equation.  $\square$

From the preceding discussion, it is already implied that the vectors  $u, v$  defined by our formulas from Theorem 2.5 and 2.6 are dual feasible. Remarkably, feasibility is preserved even when the indices  $j_i$  (respectively  $i_j$ ) do not encode the staircase path traced by Algorithm 1. In fact, for any arbitrary choice of indices  $(j_i)_{i \in [p]}$  in  $[q]$  (and analogously  $(i_j)_{j \in [q]}$  in  $[p]$ ),  $(u, v)$  remains a feasible solution for the dual (3). We conclude this section by proving this result.

**Lemma 2.8.** *Let  $(j_i)_{i \in [p]}$  be any sequence of indices in  $[q]$ . Using these indices, define vectors  $u \in \mathbb{R}^p$  and  $v \in \mathbb{R}^q$  according to (4) and (5), respectively. Then,  $(u, v)$  is feasible for (3), that is,  $u_i + v_j \leq c_{ij}$  for all  $i \in [p]$ ,  $j \in [q]$ .*

*An analogous result holds for the formulas (6) and (7) with indices  $(i_j)_{j \in [q]}$  in  $[p]$ .*

*Proof.* Fix  $i \in [p]$  and  $j \in [q]$ . With the identity  $c_{ij} = c_{1j} + \sum_{k=2}^i (c_{kj} - c_{k-1,j})$ , we obtain

$$\begin{aligned}
c_{ij} - (u_i + v_j) &= c_{1j} + \sum_{k=2}^i (c_{kj} - c_{k-1,j}) - \left( \sum_{k=2}^i (c_{kj_k} - c_{k-1,j_k}) + \right. \\
&\quad \left. c_{1j} + \sum_{\substack{k \in [p], k \geq 2 \\ j_k \leq j}} (c_{kj} - c_{k-1,j} - c_{kj_k} + c_{k-1,j_k}) \right) \\
&= \sum_{k=2}^i (c_{kj} - c_{k-1,j} - c_{kj_k} + c_{k-1,j_k}) - \sum_{\substack{k \in [p], k \geq 2 \\ j_k \leq j}} (c_{kj} - c_{k-1,j} - c_{kj_k} + c_{k-1,j_k}) \\
&= \sum_{\substack{k \in [i], k \geq 2 \\ j_k > j}} \underbrace{(c_{kj} - c_{k-1,j} - c_{kj_k} + c_{k-1,j_k})}_{\geq 0} - \sum_{\substack{k \in [p], k > i \\ j_k < j}} \underbrace{(c_{kj} - c_{k-1,j} - c_{kj_k} + c_{k-1,j_k})}_{\leq 0} \\
&\geq 0.
\end{aligned}$$

The cancellation in the third equality follows by partitioning the indices: terms with  $2 \leq k \leq i$  and  $j_k \leq j$

appear in both sums and subtract to zero. The final inequality holds because the Monge property implies that  $c_{kj} - c_{k-1,j} - c_{kj_k} + c_{k-1,j_k}$  is non-negative when  $j_k > j$  and non-positive when  $j_k < j$ .  $\square$

### 3 Application to the Discrete Ordered Median Problem

We now apply the formulas derived in Section 2. First, in Subsection 3.1, we introduce all notation needed to present the DOMP and reproduce a specific DOMP formulation by Labbé et al. (2017). In Subsection 3.2, we show how the assumption of non-increasing objective function weights allows us to simplify the model from the literature. By applying Benders decomposition to the resulting formulation, we derive two new reformulations of the DOMP for non-increasing objective weight vectors. Here, the formulas from the previous section play the key role in obtaining explicit Benders cut expressions. We conclude the section with theoretical and practical remarks (Subsections 3.3 and 3.4).

#### 3.1 Problem definition and formulation from the literature

Let  $n \in \mathbb{N}$  be the number of client locations and assume without loss of generality that new facilities can be placed exactly at these locations. Furthermore, let  $p \in \mathbb{N}$ ,  $p \leq n$  be the number of facilities that is to be opened. Next, let  $c \in \mathbb{R}_{\geq 0}^{n \times n}$  be the cost matrix, where entry  $c_{ij}$  indicates the cost of satisfying all the demand of client  $i$  from facility  $j$ . Every customer should be assigned to and fully served by the closest open facility. This implicitly assumes that there are no capacity limits for the opened facilities.

Before explaining the DOMP objective, we recall the definition of the general ordered median function.

**Definition 3.1** (Ordered median function (Nickel & Puerto, 2005)). *For a given  $x \in \mathbb{R}^n$ , let  $x_{\leq} := (x_{(1)}, x_{(2)}, \dots, x_{(n)})$  be a re-ordering of  $x$  such that  $x_{(1)} \leq x_{(2)} \leq \dots \leq x_{(n)}$ . A function  $f : \mathbb{R}^n \rightarrow \mathbb{R}$  is defined to be an ordered median function, if there exists a vector  $\lambda \in \mathbb{R}^n$  such that*

$$f(x) = \langle \lambda, x_{\leq} \rangle = \sum_{i=1}^n \lambda_i x_{(i)} \quad \forall x \in \mathbb{R}^n.$$

The DOMP objective function is an ordered median function applied to the vector of customer allocation costs. More precisely, let  $Y \subseteq [n]$  be the set of opened facilities and denote the allocation cost for each customer  $i$  by  $c_i(Y) = \min_{j \in Y} c_{ij}$ . Then,  $c(Y) := (c_i(Y))_{i \in [n]}$  is the allocation cost vector that is to be minimized in the ordered median objective. That is, the DOMP with weight vector  $\lambda \in \mathbb{R}^n$  and associated ordered median function  $f_{\lambda}$  can be stated as

$$\min_{\substack{Y \subseteq [n] \\ |Y|=p}} f_{\lambda}(c(Y)). \quad (8)$$

Note that we allow a slight overlap between the notation used for the TP and the DOMP in order to adhere to standard notation (e.g., the cost matrix  $c$ ); the meaning will always be clear from the context.

To facilitate the presentation of our models, we will make use of the following notation later in this section. For a vector  $x \in \mathbb{R}^d$ , define  $\Delta_k^x := x_k - x_{k-1}$  for  $k \in [d]$  and  $x_0 := 0$ .

In the literature, one can find plenty of different ways to model (8) as a MILP (e.g., Labbé et al., 2017; Marín et al., 2020; Ljubić et al., 2024). Many of the best-performing models are based on two-index variables  $x_{ij} \in \{0, 1\}$  taking value 1 if and only if client  $i$  is served by facility  $j$ . Furthermore, these models typically use indicator variables  $y_j \in \{0, 1\}$  for each facility  $j$  that are equal to 1 if and only if facility  $j$  is opened. Any

feasible solution to the considered ordered median location problem must lie inside the  $p$ -median polytope which is defined by

$$X := \left\{ x \in [0, 1]^{n \times n}, y \in [0, 1]^n \mid \sum_{j=1}^n y_j = p; \sum_{j=1}^n x_{ij} = 1 \forall i \in [n]; x_{ij} \leq y_j \forall i, j \in [n]; \sum_{\substack{j \in [n] \\ c_{ij} > c_{im}}} x_{ij} + y_m \leq 1 \forall i, m \in [n] \right\}.$$

The inequalities defining  $X$  ensure that exactly  $p$  facilities are opened and that each client is assigned to exactly one open facility, such that there exists no closer open facility. In the following, we will write  $X_I := X \cap (\mathbb{Z}^{n \times n} \times \mathbb{Z}^n)$  for the intersection of the  $p$ -median polytope with the integer lattice points.

This paper focuses on the “staircase model” proposed in Labbé et al. (2017), which we will refer to as  $\text{DOMP}_s$  below. In order to present their formulation, Labbé et al. (2017) sort the  $g$  distinct positive entries of the allocation cost matrix  $c$  increasingly and insert an additional zero as the first element. In particular, they write  $0 =: c_{(0)} < c_{(1)} < \dots < c_{(g)} = \max_{i,j \in [n]} c_{ij}$ . The model makes use of binary variables  $w_{lh}$  that indicate whether the allocation cost in position  $l$  is equal to  $c_{(h)}$  or not for each  $l \in [n]$  and  $h \in [g] \cup \{0\}$ . For convenience, define  $[g]_0 := [g] \cup \{0\}$ .

$$\text{DOMP}_s : \min \sum_{h=0}^g \sum_{l=1}^n \lambda_l c_{(h)} w_{lh} \quad (9a)$$

$$\text{s.t.} \quad \sum_{h=0}^g w_{lh} = 1 \quad \forall l \in [n] \quad (9b)$$

$$\sum_{l=1}^n w_{lh} = \sum_{\substack{i,j \in [n] \\ c_{ij} = c_{(h)}}} x_{ij} \quad \forall h \in [g]_0 \quad (9c)$$

$$\sum_{h' < h} w_{l+1, h'} + \sum_{h' \geq h} w_{lh'} \leq 1 \quad \forall l \in [n-1], h \in [g] \quad (9d)$$

$$w_{lh} \in \{0, 1\} \quad \forall l \in [n], h \in [g]_0 \quad (9e)$$

$$(x, y) \in X_I \quad (9f)$$

Constraints (9b) ensure that each position  $l$  in the ordered allocation cost sequence will be occupied by exactly one value  $c_{(h)}$ . By constraints (9c), the number of occurrences of each  $c_{(h)}$  in the ordered cost sequence will exactly correspond to the number of clients whose allocation cost equals  $c_{(h)}$ . Next, constraints (9d) ensure that the allocation cost sequence given by the variables  $w_{lh}$  is sorted non-decreasingly. Due to the correct definition of the variables  $w_{lh}$  by means of constraints (9b) – (9e), the objective function (9a) correctly computes the ordered  $\lambda$ -weighted sum of allocation costs which is to be minimized.

### 3.2 Benders reformulations for non-increasing objective vectors

For our application of the TP, we study non-increasing objective vectors  $\lambda$ . We start by observing the possibility to simplify model (9) in that case.

**Lemma 3.2.** *For any  $\lambda \in \mathbb{R}^n$  satisfying  $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_n$ , the following compact model is a valid  $\text{DOMP}$*

formulation.

$$mDOMP_s : \min \sum_{h=0}^g \sum_{l=1}^n \lambda_l c_{(h)} w_{lh} \quad (10a)$$

$$\text{s.t.} \quad \sum_{h=0}^g w_{lh} = 1 \quad \forall l \in [n] \quad (10b)$$

$$\sum_{l=1}^n w_{lh} = \sum_{\substack{i,j \in [n] \\ c_{ij} = c_{(h)}}} x_{ij} \quad \forall h \in [g]_0 \quad (10c)$$

$$w_{lh} \geq 0 \quad \forall l \in [n], h \in [g]_0 \quad (10d)$$

$$(x, y) \in X_I \quad (10e)$$

*Proof.* First, we show that the special structure of  $\lambda$  renders the ordering constraints (9d) redundant. It is well-known that the scalar product of a non-increasing vector  $\lambda$  with a vector  $x$  rearranged according to some permutation  $\pi$  is minimal if  $\pi$  sorts  $x$  non-decreasingly (Hardy et al., 1934, Inequality 368). More precisely, if  $x_\pi$  denotes the reordering of  $x \in \mathbb{R}^n$  according to the permutation  $\pi : [n] \rightarrow [n]$  and  $x_\leq$  denotes the non-decreasingly sorted  $x$ , then it holds  $\langle \lambda, x_\leq \rangle \leq \langle \lambda, x_\pi \rangle$  for every permutation  $\pi : [n] \rightarrow [n]$ . Now, let  $(w^*, x^*, y^*)$  be an optimal solution to (9) excluding constraints (9d) and consider the vector of optimal allocation costs  $(c_{(h_l)})_{l \in [n]}$ , where  $h_l \in [g]_0$  for  $l \in [n]$  is such that  $w_{lh_l}^* = 1$ . The existence of such  $h_l$  for each  $l \in [n]$  is guaranteed by constraints (9b) and (9e). According to the above stated rearrangement inequality, the minimization automatically ensures that  $c_{(h_l)} \leq c_{(h_{l+1})}$  and thus,  $h_l \leq h_{l+1}$  for each  $l \in [n-1]$ . So, for every  $l \in [n-1]$  and  $h \leq h_l$ , we have  $\sum_{h' < h} w_{l+1,h'}^* = 0$  and for  $h > h_l$ , it holds that  $\sum_{h' \geq h} w_{l,h'}^* = 0$ . Hence, constraints (9d) are all satisfied.

Next, the upper bounds  $w_{lh} \leq 1$  for every  $l \in [n]$  and  $h \in [g]_0$  are implied by (9b), so (9e) can be replaced by  $w_{lh} \geq 0$ ,  $w_{lh} \in \mathbb{Z}$  for all  $l \in [n]$ ,  $h \in [g]_0$ .

Finally, the integrality restrictions on the  $w$ -variables can be relaxed. Consider the subproblem (10a) – (10d) for any fixed  $(\bar{x}, \bar{y}) \in X_I$ . Since  $\sum_{h=0}^g \sum_{\substack{i,j \in [n] \\ c_{ij} = c_{(h)}}} \bar{x}_{ij} = \sum_{i,j \in [n]} \bar{x}_{ij} = n$ , this is a balanced TP with integer supplies and demands. As mentioned in Section 2, such problems are feasible, bounded, and every basic feasible solution is integral.

Overall, we see that model (10) always admits an optimal solution  $(w^*, x^*, y^*)$  which is feasible for model (9). Since the feasible region of (9) is a subset of the feasible region of (10), the solution  $(w^*, x^*, y^*)$  is optimal for (9).  $\square$

Following an approach presented by Ljubić et al. (2024), we can derive a Benders reformulation of the staircase model (10) for non-increasing  $\lambda$  by projecting out the variables  $w$  and replacing the objective function by a new variable  $\theta$ . Let  $(\bar{x}, \bar{y}) \in X_I$  and define  $\bar{x}_h := \sum_{\substack{i,j \in [n] \\ c_{ij} = c_{(h)}}} \bar{x}_{ij}$  for each  $h \in [g]_0$ . Then, we are facing the following subproblem.

$$\min \sum_{h=0}^g \sum_{l=1}^n \lambda_l c_{(h)} w_{lh} \quad (11a)$$

$$\text{s.t.} \quad \sum_{h=0}^g w_{lh} = 1 \quad \forall l \in [n] \quad (11b)$$

$$\sum_{l=1}^n w_{lh} = \bar{x}_h \quad \forall h \in [g]_0 \quad (11c)$$

$$w_{lh} \geq 0 \quad \forall l \in [n], h \in [g]_0 \quad (11d)$$

As already observed in the proof of Lemma 3.2, this is a balanced TP with integral right-hand sides, which is feasible and bounded. By introducing dual variables  $u_l$  for constraints (11b) and  $v_h$  for (11c), the dual LP is as follows.

$$\max \quad \sum_{l=1}^n u_l + \sum_{h=0}^g \bar{x}_h v_h \quad (12a)$$

$$\text{s.t.} \quad u_l + v_h \leq \lambda_l c_{(h)} \quad \forall l \in [n], h \in [g]_0 \quad (12b)$$

$$u_l, v_h \in \mathbb{R} \quad \forall l \in [n], h \in [g]_0 \quad (12c)$$

For any dual feasible solution  $(u, v)$ , we obtain a Benders cut of the form

$$\theta \geq \sum_{l=1}^n u_l + \sum_{h=0}^g v_h \sum_{\substack{i,j \in [n] \\ c_{ij} = c_{(h)}}} x_{ij}. \quad (13)$$

Observe that the matrix  $(\lambda_l c_{(h)})_{l \in [n], h \in [g]_0}$  satisfies the Monge property: for  $1 \leq l < l' \leq n$  and  $0 \leq h < h' \leq g$ , we see that  $\lambda_l c_{(h')} + \lambda_{l'} c_{(h)} - (\lambda_l c_{(h)} + \lambda_{l'} c_{(h')}) = (\lambda_l - \lambda_{l'})(c_{(h')} - c_{(h)}) \geq 0$ . Thus,  $\lambda_l c_{(h)} + \lambda_{l'} c_{(h')} \leq \lambda_l c_{(h')} + \lambda_{l'} c_{(h)}$  holds true.

Therefore, the theory from Section 2 applies. Particularly, the formulas given in Theorem 2.5 and Theorem 2.6 yield optimal dual variables  $u$  and  $v$  that can be inserted into the Benders optimality cut expression (13). Using Theorem 2.5, we obtain the following exponential-sized model for DOMP with non-increasing objective vector.

**Theorem 3.3.** *For any  $\lambda \in \mathbb{R}^n$  satisfying  $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_n$ , the following exponential-sized model is a valid DOMP formulation.*

$$mDOMP_{sB1} : \min \quad \theta \quad (14a)$$

$$\text{s.t.} \quad \theta \geq \sum_{k=1}^n \Delta_k^\lambda \left( (n-k+1)c_{(f(k))} + \sum_{\substack{i,j \in [n] \\ c_{ij} > c_{(f(k))}}} (c_{ij} - c_{(f(k))})x_{ij} \right) \quad \begin{array}{l} \forall f : [n] \rightarrow [g]_0 \text{ mon. incr.} \\ \text{with } f(1) = 0 \end{array} \quad (14b)$$

$$\theta \in \mathbb{R} \quad (14c)$$

$$(x, y) \in X_I \quad (14d)$$

*Proof.* For a monotonically increasing mapping  $f : [n] \rightarrow [g]_0$  with  $f(1) = 0$ , define the vectors  $u^f, v^f$  as

follows:

$$\begin{aligned} u_l^f &= \sum_{k=2}^l (\lambda_k - \lambda_{k-1}) c_{(f(k))} & l \in [n], \\ v_h^f &= \sum_{\substack{k \in [n] \\ f(k) < h}} (\lambda_k - \lambda_{k-1}) (c_{(h)} - c_{(f(k))}) & h \in [g]_0. \end{aligned} \quad (15)$$

Note that  $v_h^f = \lambda_1 c_{(h)} + \sum_{\substack{k \in [n], k \geq 2 \\ f(k) \leq h}} (\lambda_k - \lambda_{k-1}) (c_{(h)} - c_{(f(k))})$  since  $\lambda_1 c_{(h)} = (\lambda_1 - \lambda_0) (c_{(h)} - c_{(f(1))})$  and  $c_{(h)} - c_{(f(k))} = 0$  whenever  $f(k) = h$ . So, the above formulas (15) correspond to those given in Theorem 2.5 for the optimal dual variables  $(u, v)$  and  $f(k)$  corresponds to the indices  $j_i$  encoding a potential staircase path that Algorithm 1 may traverse. By Lemma 2.8,  $(u^f, v^f)$  is a feasible solution to (12). Furthermore, since every valid staircase path translates to a monotonically increasing function  $f$ , no cuts are omitted in (14b).

Therefore, by Theorem 2.5 and the Benders optimality cut formula (13), it remains to show that for any monotonically increasing  $f : [n] \rightarrow [g]_0$  with  $f(1) = 0$ , it holds

$$\sum_{l=1}^n u_l^f + \sum_{h=0}^g v_h^f \sum_{\substack{i, j \in [n] \\ c_{ij} = c_{(h)}}} x_{ij} = \sum_{k=1}^n \Delta_k^\lambda \left( (n - k + 1) c_{(f(k))} + \sum_{\substack{i, j \in [n] \\ c_{ij} > c_{(f(k))}}} (c_{ij} - c_{(f(k))}) x_{ij} \right). \quad (16)$$

Through rearrangement of terms, it is straightforward to verify that

$$\sum_{l=1}^n u_l^f = \sum_{k=1}^n \Delta_k^\lambda (n - k + 1) c_{(f(k))}. \quad (17)$$

Furthermore, we can rewrite

$$\begin{aligned} \sum_{h=0}^g v_h^f \sum_{\substack{i, j \in [n] \\ c_{ij} = c_{(h)}}} x_{ij} &= \sum_{h=0}^g \sum_{\substack{k \in [n] \\ f(k) < h}} \Delta_k^\lambda (c_{(h)} - c_{(f(k))}) \sum_{\substack{i, j \in [n] \\ c_{ij} = c_{(h)}}} x_{ij} \\ &= \sum_{k=1}^n \Delta_k^\lambda \sum_{\substack{h \in [g]_0 \\ h > f(k)}} \sum_{\substack{i, j \in [n] \\ c_{ij} = c_{(h)}}} (c_{(h)} - c_{(f(k))}) x_{ij} \\ &= \sum_{k=1}^n \Delta_k^\lambda \sum_{\substack{i, j \in [n] \\ c_{ij} > c_{(f(k))}}} (c_{ij} - c_{(f(k))}) x_{ij}. \end{aligned} \quad (18)$$

In the last equation, we used that  $c_{(\cdot)}$  is strictly increasing:  $h > f(k)$  and  $c_{ij} = c_{(h)}$  imply that  $c_{ij} > c_{(f(k))}$ . Conversely, if  $c_{ij} > c_{(f(k))}$ , then there exists  $h \in [g]_0$ ,  $h > f(k)$  such that  $c_{ij} = c_{(h)}$ .

Finally, adding (17) and (18) proves (16).  $\square$

If we use the formulas from Theorem 2.6 instead of the ones from Theorem 2.5, we obtain another exponential-sized model for DOMP with non-increasing objective vector.

**Theorem 3.4.** *For any  $\lambda \in \mathbb{R}^n$  satisfying  $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_n$ , the following exponential-sized model is a*

valid DOMP formulation.

$$mDOMP_{sB2} : \min \quad \theta \tag{19a}$$

$$\begin{aligned} \text{s.t.} \quad \theta \geq & \sum_{k=0}^{g-1} \Delta_{k+1}^{c(\cdot)} \left( \sum_{\substack{l \in [n] \\ l > f(k)}} (\lambda_l - \lambda_{f(k)}) \right. \\ & \left. + \lambda_{f(k)} \sum_{\substack{i,j \in [n] \\ c_{ij} > c(k)}} x_{ij} \right) \quad \forall f : [g-1]_0 \rightarrow [n] \text{ mon. incr.} \end{aligned} \tag{19b}$$

$$\theta \in \mathbb{R} \tag{19c}$$

$$(x, y) \in X_I \tag{19d}$$

*Proof.* For a monotonically increasing mapping  $f : [g-1]_0 \rightarrow [n]$ , define the vectors  $u^f, v^f$  as follows:

$$\begin{aligned} u_l^f &= \sum_{\substack{k \in [g-1]_0 \\ f(k) < l}} (\lambda_l - \lambda_{f(k)}) (c_{(k+1)} - c_{(k)}) \quad l \in [n], \\ v_h^f &= \sum_{k=0}^{h-1} \lambda_{f(k)} (c_{(k+1)} - c_{(k)}) \quad h \in [g]_0. \end{aligned} \tag{20}$$

A simple index shift shows that  $u_l^f = \lambda_l c_{(0)} + \sum_{\substack{k \in [g] \\ f(k-1) \leq l}} (\lambda_l - \lambda_{f(k-1)}) (c_{(k)} - c_{(k-1)})$  and that  $v_h^f = \sum_{k=1}^h \lambda_{f(k-1)} (c_{(k)} - c_{(k-1)})$ . So, the above formulas (20) correspond exactly to those given in Theorem 2.6 for the optimal dual variables  $(u, v)$  and  $f(k)$  corresponds to the indices  $i_j$  encoding a potential staircase path that Algorithm 1 may traverse. By Lemma 2.8,  $(u^f, v^f)$  is a feasible solution to (12). With the same argument as in the proof of Theorem 3.3, the set of cuts in (19b) is complete.

Therefore, by Theorem 2.6 and the Benders optimality cut formula (13), it remains to show that for any monotonically increasing  $f : [g]_0 \rightarrow [n]$  with  $f(g) = n$ , it holds

$$\sum_{l=1}^n u_l^f + \sum_{h=0}^g v_h^f \sum_{\substack{i,j \in [n] \\ c_{ij} = c(h)}} x_{ij} = \sum_{k=0}^{g-1} \Delta_{k+1}^{c(\cdot)} \left( \sum_{\substack{l \in [n] \\ l > f(k)}} (\lambda_l - \lambda_{f(k)}) + \lambda_{f(k)} \sum_{\substack{i,j \in [n] \\ c_{ij} > c(k)}} x_{ij} \right). \tag{21}$$

By definition of  $u_l^f$ , it holds that

$$\sum_{l=1}^n u_l^f = \sum_{k=0}^{g-1} \Delta_{k+1}^{c(\cdot)} \sum_{\substack{l \in [n] \\ l > f(k)}} (\lambda_l - \lambda_{f(k)}). \tag{22}$$

Furthermore, we can rewrite

$$\begin{aligned} \sum_{h=0}^g v_h^f \sum_{\substack{i,j \in [n] \\ c_{ij} = c(h)}} x_{ij} &= \sum_{h=0}^g \sum_{k=0}^{h-1} \lambda_{f(k)} (c_{(k+1)} - c_{(k)}) \sum_{\substack{i,j \in [n] \\ c_{ij} = c(h)}} x_{ij} \\ &= \sum_{h=1}^g \lambda_{f(h-1)} (c_{(h)} - c_{(h-1)}) \sum_{h'=h}^g \sum_{\substack{i,j \in [n] \\ c_{ij} = c(h')}} x_{ij} \end{aligned}$$

$$\begin{aligned}
&= \sum_{h=0}^{g-1} \lambda_{f(h)} (c_{(h+1)} - c_{(h)}) \sum_{h'=h+1}^g \sum_{\substack{i,j \in [n] \\ c_{ij} = c_{(h')}} x_{ij} \\
&= \sum_{h=0}^{g-1} \lambda_{f(h)} (c_{(h+1)} - c_{(h)}) \sum_{\substack{i,j \in [n] \\ c_{ij} > c_{(h)}}} x_{ij}.
\end{aligned} \tag{23}$$

In the last equation, we used that  $c_{(\cdot)}$  is strictly increasing. More precisely,  $h' > h$  and  $c_{ij} = c_{(h')}$  imply that  $c_{ij} > c_{(h)}$ . Conversely, if  $c_{ij} > c_{(h)}$ , then there exists  $h' \in [g]_0$ ,  $h' > h$  such that  $c_{ij} = c_{(h')}$ .

Finally, adding (22) and (23) proves (21).  $\square$

### 3.3 Theoretical notes

It is clear that both  $\text{mDMP}_{sB1}$  and  $\text{mDMP}_{sB2}$  involve exponentially many constraints. A closer inspection shows that the number of constraints is the same for both models.

**Lemma 3.5.** *The number of constraints of type (14b) and (19b) is  $\binom{n+g-1}{n-1}$ .*

*Proof.* Using standard counting techniques, one can show that the number of monotonically increasing functions  $f : [a] \rightarrow [b]$  is equal to  $\binom{a+b-1}{a}$  (see, e.g., Stanley, 1997). For (14b), we have  $a = n - 1$  and  $b = g + 1$ , so the number of constraints is  $\binom{n+g-1}{n-1}$ . Analogously, with  $a = g$  and  $b = n$ , we have  $\binom{n+g-1}{g}$  constraints of type (19b). Now, the claim follows from the well-known identity  $\binom{N}{k} = \binom{N}{N-k}$  for  $0 \leq k \leq N$ .  $\square$

Recall from Section 2 that the dual solutions given by Theorem 2.5 and Theorem 2.6 coincide up to a shift by  $\lambda_1 c_{(0)} = 0$  (cf. Lemma 2.4). So, in fact, the two formulas yield identical values in our application. This, together with Lemma 3.5 is a strong indicator that the models  $\text{mDMP}_{sB1}$  and  $\text{mDMP}_{sB2}$  are actually identical. Indeed, this is what we show in the next lemma.

**Lemma 3.6.** *The constraint sets (14b) and (19b) are identical.*

*Proof.* Let  $f^1 : [n] \rightarrow [g]_0$ ,  $f^1(1) = 0$  be a monotonically increasing function. This function defines the constraint of type (14b) of the form  $\theta \geq \sum_{l=1}^n u_l^{f^1} + \sum_{h=0}^g v_h^{f^1} \sum_{\substack{i,j \in [n] \\ c_{ij} = c_{(h)}}} x_{ij}$ , where  $u^{f^1}$  and  $v^{f^1}$  are defined according to (15). We can associate a unique staircase path on the  $n \times (g+1)$  grid that connects the northwest with the southeast corner: consider the path  $\mathcal{A}$  from  $(1, 0)$  to  $(n, g)$  that contains all points  $(l, f^1(l))$  for  $l \in [n]$  such that  $f^1(l) = \min\{h \in [g]_0 \mid (l, h) \in \mathcal{A}\}$ . This path has the form

$$\begin{aligned}
\mathcal{A} = \{ & (1, 0), \dots, (1, f^1(2)), (2, f^1(2)), \dots, (2, f^1(3)), (3, f^1(3)), \dots \\
& \dots, (n-1, f^1(n)), (n, f^1(n)), \dots, (n, g) \}.
\end{aligned}$$

Define  $f^2 : [g-1]_0 \rightarrow [n]$  as it was done earlier in Section 2 by  $f^2(h) := \max\{l \in [n] \mid (l, h) \in \mathcal{A}\}$  for every  $h \in [g-1]_0$ . By the staircase structure of  $\mathcal{A}$ , we see that  $f^2$  is monotonically increasing. Therefore, it defines the constraint of type (19b) of the form  $\theta \geq \sum_{l=1}^n u_l^{f^2} + \sum_{h=0}^g v_h^{f^2} \sum_{\substack{i,j \in [n] \\ c_{ij} = c_{(h)}}} x_{ij}$ , where  $u^{f^2}$  and  $v^{f^2}$  are defined according to (20). Lemma 2.4 implies that  $(u^{f^2}, v^{f^2}) = (u^{f^1} + \lambda_1 c_{(0)}, v^{f^1} - \lambda_1 c_{(0)}) = (u^{f^1}, v^{f^1})$ . As a result, the constraints induced by  $f^1$  and  $f^2$  are identical.

Conversely, let  $f^2 : [g-1]_0 \rightarrow [n]$  be a monotonically increasing function. Again, this function uniquely describes a staircase path connecting the northwest and the southeast corner, namely the path  $\mathcal{A}$  from  $(1, 0)$

to  $(n, g)$  containing all points  $(f^2(h), h)$  for  $h \in [g-1]_0$  such that  $f^2(h) = \max\{l \in [n] \mid (l, h) \in \mathcal{A}\}$ . It has the form

$$\mathcal{A} = \{(1, 0), \dots, (f^2(0), 0), (f^2(0), 1), \dots, (f^2(1), 1), (f^2(1), 2), \dots, (f^2(g-1), g-1), (f^2(g-1), g), \dots, (n, g)\}.$$

For  $f^1 : [n] \rightarrow [g]_0$  with  $f^1(1) := 0$  and  $f^1(l) := \min\{h \in [g]_0 \mid (l, h) \in \mathcal{A}\}$  for every  $l \in \{2, \dots, n\}$  we again find that  $f^2$  and  $f^1$  induce the exact same constraint.  $\square$

Even though  $\text{mDOMP}_{sB1}$  and  $\text{mDOMP}_{sB2}$  describe the same polytope, they differ from a computational perspective, which can make one formulation preferable to the other. This issue is discussed in Section 3.4 and revisited in Section 4.3.

### 3.4 Practical notes

Since there are exponentially many constraints of type (14b) and (19b), they need to be separated when solving the models  $\text{mDOMP}_{sB1}$  and  $\text{mDOMP}_{sB2}$  in practice. Given an integer solution  $(\bar{x}, \bar{y}) \in X_I$  at some node of the branch-and-bound (B&B) tree, one needs to identify the corresponding monotonically increasing function  $f$  that encodes the staircase path traversed by the northwest-corner rule. If the objective function value  $\bar{\theta}$  at that node is less than  $\sum_{l=1}^n u_l^f + \sum_{h=0}^g v_h^f \sum_{\substack{i,j \in [n] \\ c_{ij} = c(h)}} \bar{x}_{ij}$  minus a cut violation tolerance  $\varepsilon \geq 0$ , then the constraint  $\bar{\theta} \geq \sum_{l=1}^n u_l^f + \sum_{h=0}^g v_h^f \sum_{\substack{i,j \in [n] \\ c_{ij} = c(h)}} x_{ij}$  is added to the model. Algorithm 4 summarizes this procedure.

---

**Algorithm 4:** Benders cut separation for  $\text{mDOMP}_{sB1}$  and  $\text{mDOMP}_{sB2}$

---

- 1 Consider an integer solution  $(\bar{x}, \bar{y}) \in X_I$  with objective value  $\bar{\theta}$  at some B&B node
  - 2 Identify the monotonically increasing  $f$  corresponding to  $(\bar{x}, \bar{y})$
  - 3 Compute the vectors  $u^f$  and  $v^f$
  - 4 Set  $\text{RHS} \leftarrow \sum_{l=1}^n u_l^f + \sum_{h=0}^g v_h^f \sum_{\substack{i,j \in [n] \\ c_{ij} = c(h)}} \bar{x}_{ij}$
  - 5 **if**  $\bar{\theta} < \text{RHS} - \varepsilon$  **then**
  - 6     Add the cut  $\theta \geq \sum_{l=1}^n u_l^f + \sum_{h=0}^g v_h^f \sum_{\substack{i,j \in [n] \\ c_{ij} = c(h)}} x_{ij}$
- 

Next, we describe how to obtain the function  $f$  from a given solution  $(\bar{x}, \bar{y}) \in X_I$  for both cases  $\text{mDOMP}_{sB1}$  and  $\text{mDOMP}_{sB2}$ . The first step is to find the indices  $h_l$  or  $l_h$ , respectively, that encode the solution produced by Algorithm 1 on subproblem (11). These indices are given by Corollary 2.7.

(i) For  $l \in \{2, \dots, n\}$ ,  $h_l = \min\{h \in [g]_0 \mid l-1 \leq \sum_{k=0}^h \bar{x}_k\}$ .

(ii) For  $h \in \{0, \dots, g-1\}$ ,  $l_h = \max\{l \in [n] \mid l-1 \leq \sum_{k=0}^h \bar{x}_k\} = \min\{1 + \sum_{k=0}^h \bar{x}_k, n\}$ .

Then, one may set  $f(1) = 0$ ,  $f(l) = h_l$  for  $l \in \{2, \dots, n\}$  or  $f(h) = l_h$  for  $h \in \{0, \dots, g-1\}$ , respectively.

As mentioned in Section 3.3, it does not matter theoretically which formulas are used to compute  $u$  and  $v$ . However, according to (i) and (ii) there are obvious computational differences when calculating the indices  $h_l$  and  $l_h$ . Moreover, the sums in the formulas given by Theorem 2.5 and Theorem 2.6 may differ significantly in their number of terms, which also affects the efficiency of a numerical evaluation. For these reasons, we tested both variants in our numerical study.

Notice that solving the transportation subproblem directly with the northwest-corner rule and retrieving the duals using Algorithm 2 or 3 results in a time-complexity of  $\mathcal{O}(n + g)$ , while any straightforward implementation of Algorithm 4 runs in  $\mathcal{O}(ng)$ . Nevertheless, as most of the computational time in our experiments is spent solving LP relaxations within the B&B process rather than performing the separation procedure, this difference has only a minor impact on the overall runtime. To highlight the practical differences between  $\text{mDOMP}_{sB1}$  and  $\text{mDOMP}_{sB2}$ , we therefore use Algorithm 4 in the numerical experiments.

Lastly, for a proper initialization, one must add a generally valid lower bound on  $\theta$  to each model. To obtain such a bound, compute  $c_i^{\min} = \min\{c_{ij} \mid j \in [n]\}$  and  $c_i^{\max} = \max\{c_{ij} \mid j \in [n]\}$  for  $i \in [n]$  and sort  $c^{\min}$  and  $c^{\max}$  non-decreasingly. Denoting the sorted vectors by  $c_{(\cdot)}^{\min}$  and  $c_{(\cdot)}^{\max}$ , it is easy to see that the objective function value always satisfies

$$\theta \geq \sum_{i \in [n]: \lambda_i \geq 0} \lambda_i c_{(i)}^{\min} + \sum_{i \in [n]: \lambda_i < 0} \lambda_i c_{(i)}^{\max}.$$

This bound is independent of any model and therefore works for both (14) and (19).

## 4 Numerical experiments

In this section, we present the results of our numerical experiments which we conducted to test  $\text{mDOMP}_{sB1}$  and  $\text{mDOMP}_{sB2}$ . The code was written in Python 3.12 and Gurobi 12.0.3 (Gurobi Optimization, LLC, 2026) was chosen as our MILP solver. For an unbiased comparison of the models, we disabled presolve, heuristics, as well as solver generated cuts, and we restricted the number of threads to one, following the settings in Ljubić et al. (2024). Regarding the termination criteria, a time limit of two hours (7200 seconds) and a MIP Gap of  $10^{-6}$  per solve were set. Detailed information on these criteria can be found in Gurobi’s reference manual (Gurobi Optimization, LLC, 2026). Next, we implemented the necessary separation procedures (Algorithm 4) as described in Section 3.4 by means of Gurobi’s callback mechanism. The cut violation parameter was chosen as  $\varepsilon = 0.01$ , again complying with the choice in Ljubić et al. (2024). All computations were performed on AMD EPYC 9454 processors on the bwUniCluster system.

In the remainder of this section, we describe our test instances (Section 4.1), briefly introduce two state-of-the-art models against which we compare our new formulations (Section 4.2), and discuss our findings (Section 4.3).

### 4.1 Instances

We are testing our DOMP formulations on two sets of instances: a collection of randomly generated instances and a benchmark set taken from the literature. For the randomly generated set, the number of locations  $n$  takes the values 20, 30, 50, 100, 150, and 200. For each value of  $n$ , five  $n \times n$  cost matrices  $c$  with random entries in  $[100, 1000]$  and two decimal places are generated. The second set is based on the well-known  $p$ -median instances from OR-Library (Beasley, 1985) and will henceforth also be referred to as Beasley instances. Specifically, we use the cost matrices of sizes  $n = 100, 200$ , and  $300$  (five matrices per  $n$ , corresponding to `pmed1`, `...`, `pmed15`). Because these matrices represent shortest-path distances, they satisfy the triangle inequality. Moreover, the Beasley cost matrices are symmetric, contain many repeated entries, and have a zero diagonal, which is known as the free self-service property in the location literature. For both datasets, the number  $p$  of locations to be opened is set to  $\lfloor \frac{n}{10} \rfloor$ ,  $\lfloor \frac{n}{4} \rfloor$ ,  $\lfloor \frac{n}{3} \rfloor$ , or  $\lfloor \frac{n}{2} \rfloor$ .

The considered objective weight vectors  $\lambda$  are all non-increasing, as required by Theorems 3.3 and 3.4. They include common obnoxious facility criteria such as the obnoxious  $p$ -median ( $\lambda_1$ ),  $p$ -center ( $\lambda_2$ ),  $k$ -centrum ( $\lambda_3$ ), and range ( $\lambda_6$ ) objectives. Also, the reverse criterion ( $\lambda_7$ ) has appeared earlier in the DOMP literature for numerical testing (e.g., Domínguez & Marín, 2020; Ljubić et al., 2024). Additionally, we consider three further, less studied objectives ( $\lambda_4, \lambda_5, \lambda_8$ ) to obtain a more comprehensive picture of the models' performance. The ninth type of objective vector ( $\lambda_9$ ) is obtained by randomly generating  $n$ -vectors with entries in  $[-n, n] \cap \mathbb{Z}$  and sorting them in non-increasing sequence. Despite their limited interpretability, random objectives provide a useful benchmark, as their unstructured nature is expected to yield instances that are difficult to solve. Finally, as a canonical baseline from location theory, we also evaluate the standard  $p$ -median objective ( $\lambda_{10}$ ). Table 1 provides a summary of all tested  $\lambda$ -vectors.

| Notation       | $\lambda$ -vector                             | Name                                                      |
|----------------|-----------------------------------------------|-----------------------------------------------------------|
| $\lambda_1$    | $(-1, \dots, -1)$                             | obnoxious $p$ -median                                     |
| $\lambda_2$    | $(0, \dots, 0, -1)$                           | obnoxious $p$ -center                                     |
| $\lambda_3$    | $(0, \dots, 0, \underbrace{-1, \dots, -1}_k)$ | obnoxious $k$ -centrum, $k = \lfloor \frac{n}{2} \rfloor$ |
| $\lambda_4$    | $(\underbrace{1, \dots, 1}_k, 0, \dots, 0)$   | $k$ -min, $k = \lfloor \frac{n}{2} \rfloor$               |
| $\lambda_5$    | $(\underbrace{1, \dots, 1}_k, -1, \dots, -1)$ | obnoxious $k$ -range, $k = \lfloor \frac{n}{2} \rfloor$   |
| $\lambda_6$    | $(1, 0, \dots, 0, -1)$                        | obnoxious range                                           |
| $\lambda_7$    | $(n, n-1, \dots, 2, 1)$                       | reverse                                                   |
| $\lambda_8$    | $(-1, -2, \dots, -(n-1), -n)$                 | negative reverse                                          |
| $\lambda_9$    | $\lambda \in \{-n, -n+1, \dots, n-1, n\}^n$   | random non-increasing                                     |
| $\lambda_{10}$ | $(1, \dots, 1)$                               | $p$ -median                                               |

Table 1: Summary of tested  $\lambda$ -vectors.

Every tuple  $(n, p, c, \lambda)$  forms a single test instance. For the randomly generated dataset, this results in  $6 \cdot 4 \cdot 5 \cdot 10 = 1200$  test instances. Additionally, there are  $3 \cdot 4 \cdot 5 \cdot 10 = 600$  Beasley instances. All 1800 total instances are publicly available online (Nickel et al., 2026).

## 4.2 Reference models

We selected two benchmark models from the literature for comparison with our new formulations. Both were proposed by Ljubić et al. (2024).

The first one, referred to as  $\text{mDOMP}_{rB}$  ( $\text{DOMP}_{r0B}$  in Ljubić et al. (2024)), is a Benders reformulation of the well-known radius formulation introduced by Puerto (2008) and Marín et al. (2009), a DOMP model that is similar to the staircase formulation (9). Although  $\text{mDOMP}_{rB}$  has a polynomial number of constraints, it is advantageous to implement the separation procedure given by Ljubić et al. (2024) in order to improve the model's efficiency. A comparison with  $\text{mDOMP}_{rB}$  is particularly interesting, since it is a specialized model for non-increasing  $\lambda$ -vectors, similar to  $\text{mDOMP}_{sB1}$  and  $\text{mDOMP}_{sB2}$ . Furthermore, because the size of the formulation depends on the number of distinct entries in the cost matrix,  $\text{mDOMP}_{rB}$  is expected to perform well on datasets with many repeated cost values, such as the Beasley instances.

The second reference model, which we call  $\text{DOMP}_{\theta B}$  ( $\text{DOMP}_{\theta 1}$  in Ljubić et al. (2024)), is a Benders reformulation of a compact DOMP formulation by Marín et al. (2020). Like  $\text{mDOMP}_{rB}$ , this model has a polynomial number of constraints, but dynamic constraint separation is again used to improve its computational efficiency.  $\text{DOMP}_{\theta B}$  turned out to be one of the best-performing DOMP models in Ljubić et al. (2024) for random instances. Note that  $\text{DOMP}_{\theta B}$  is valid for any  $\lambda \in \mathbb{R}^n$  and does not require  $\lambda$  to be non-increasing.

For more details on  $\text{mDOMP}_{rB}$  and  $\text{DOMP}_{\theta B}$ , we refer the reader to Ljubić et al. (2024).

### 4.3 Discussion

In this subsection, we present and discuss some of the most relevant aspects for evaluating and comparing the performance of our proposed models. Specifically, we look at the number of solved instances alongside runtimes (Section 4.3.1), optimality gaps and bounds (Section 4.3.2), as well as the efficiency of the models' separation procedures (Section 4.3.3). The key findings are summarized in Section 4.3.4. Detailed numerical results can be found online (Nickel et al., 2026).

#### 4.3.1 Solved instances and runtime

To begin with, Figure 1 summarizes, for each model, the number of instances from the random dataset solved to optimality and the corresponding average runtime, separated by instance size. Unsurprisingly, the number

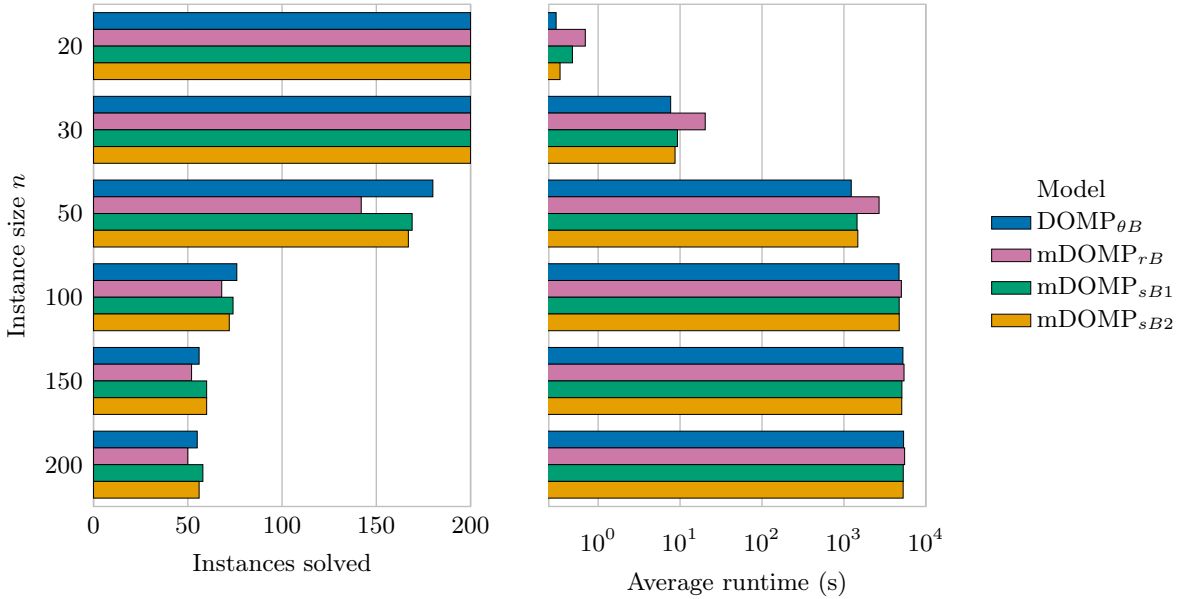


Figure 1: Number of random instances solved to optimality vs. average runtime per instance size  $n$ .

of solved instances decreases for each model as  $n$  increases, whereas the average runtimes increase with  $n$ . For small sizes ( $n \in \{20, 30\}$ ), Gurobi solved all instances within the time limit across all models. At  $n = 30$ , the average runtimes are below 10 seconds, except for  $\text{mDOMP}_{rB}$  with roughly 20 seconds. For medium-sized instances, differences between models become slightly more pronounced. At  $n = 50$ ,  $\text{mDOMP}_{rB}$  solved the fewest instances and therefore reached the time limit most frequently, resulting in the highest average runtime. In contrast,  $\text{DOMP}_{\theta B}$  solved the most instances of this size, achieving the lowest average runtime. For larger instances ( $n \in \{100, 150, 200\}$ ), only objectives  $\lambda_4$ ,  $\lambda_7$ , and  $\lambda_{10}$  are widely solved to optimality, while most other instances reach the time limit. As a result, average runtimes become similar across all models. However,  $\text{mDOMP}_{rB}$  again solves fewer of these large instances to optimality than the other models. In line with the theoretical results, models  $\text{mDOMP}_{sB1}$  and  $\text{mDOMP}_{sB2}$  achieve similar performance across all instance sizes.

Turning to the instances from the literature, Figure 2 shows the number of solved instances and the average runtimes on the Beasley dataset. For sizes  $n = 100$  and  $n = 200$ , all four formulations remain competitive. At

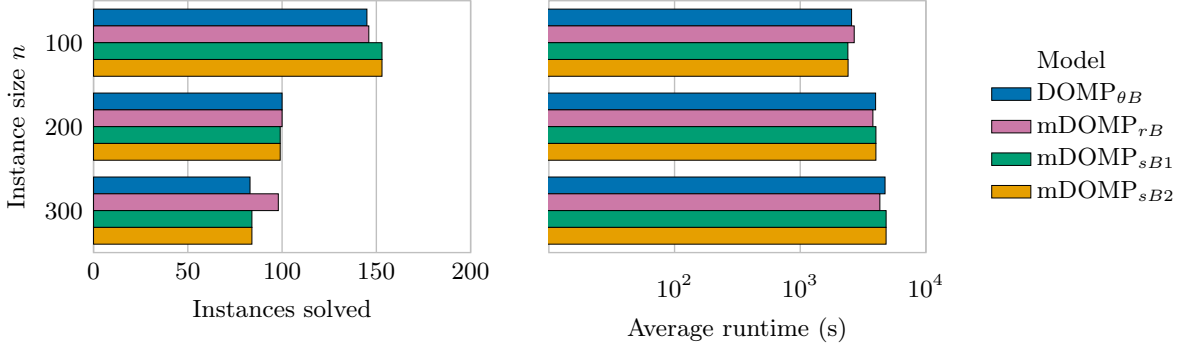


Figure 2: Number of Beasley instances solved to optimality vs. average runtime per instance size  $n$ .

$n = 100$ , mDOMP $_{sB1}$  and mDOMP $_{sB2}$  stand out by solving the highest number of instances, accompanied by slightly lower average runtimes compared to the reference models. At  $n = 200$ , all models solve roughly half of the instances within similar average times. A clear differentiation emerges at the largest size,  $n = 300$ . Here, mDOMP $_{rB}$  outperforms the other formulations, still solving nearly half (49%) of the instances of this size to optimality and recording the lowest average runtime. The remaining models solve approximately 42% for these instances.

Considering the aggregated numbers of solved instances per dataset given in Table 2, it is evident that the structured Beasley instances are computationally easier than the random instances. More precisely, for

| Dataset | DOMP $_{\theta B}$ | mDOMP $_{rB}$ | mDOMP $_{sB1}$ | mDOMP $_{sB2}$ |
|---------|--------------------|---------------|----------------|----------------|
| Random  | 131                | 118           | 132            | 128            |
| Beasley | 245                | 246           | 252            | 252            |

Table 2: Total number of instances solved to optimality for  $n \in \{100, 200\}$  (out of 400 instances per dataset: 200 for  $n = 100$  and 200 for  $n = 200$ ).

$n \in \{100, 200\}$ , all models find optimal solutions to more than 60% of the Beasley instances, whereas they solve less than one third of the random instances of these sizes to optimality.

To provide some insights on the impact of the  $\lambda$  objective vector on the computational results, Figure 3 plots the number of instances solved to optimality, disaggregated by the  $\lambda$  type. This chart reveals that the computational difficulty of an instance depends on the evaluated criterion. Clearly, the  $k$ -min ( $\lambda_4$ ), reverse ( $\lambda_7$ ), and  $p$ -median ( $\lambda_{10}$ ) objectives result in the easiest instances. Across all formulations, nearly all instances featuring these objectives were solved to optimality. For the  $p$ -median problem, this is known to be related to integer-friendliness, meaning that the LP relaxation frequently yields integer-valued solutions directly, and when it does not, only a small branching and bounding effort is needed to reach an optimum (see ReVelle, 1993). Table 3 confirms that for the  $p$ -median objective and shows the same pattern for  $\lambda_4$  and  $\lambda_7$ : the average number of B&B nodes required until the solver terminates with an optimal solution is lowest in these cases, regardless of the model. Furthermore,  $\lambda_2$  and  $\lambda_6$  can be seen as medium difficult for the tested models, since roughly two thirds of them can be solved to optimality. The remaining obnoxious criteria ( $\lambda_1, \lambda_3, \lambda_5$ ), alongside the negative reverse ( $\lambda_8$ ) and random non-increasing ( $\lambda_9$ ) objectives, pose the greatest computational challenge for all models. Certifying optimality requires high branching efforts and in most cases more than the two hour time limit. The comparison between the models shows that mDOMP $_{rB}$  falls behind on  $\lambda_1, \lambda_3, \lambda_5, \lambda_8$ , and  $\lambda_{10}$ , but outperforms the other models on  $\lambda_2$  and  $\lambda_6$  in terms of number

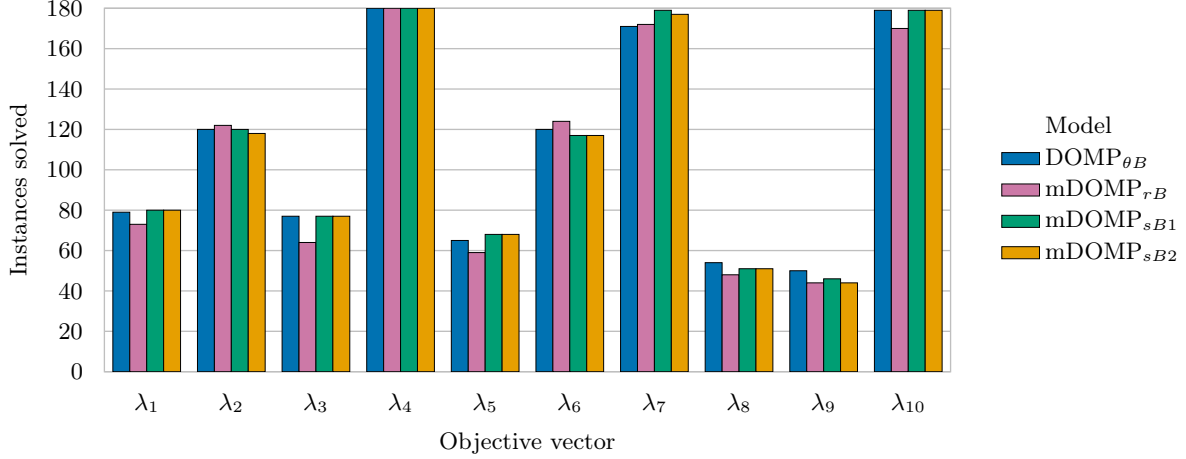


Figure 3: Total number of instances from all datasets solved to optimality per objective vector  $\lambda$ .

| $\lambda$       | DOMP <sub>θB</sub> | mDOMP <sub>rB</sub> | mDOMP <sub>sB1</sub> | mDOMP <sub>sB2</sub> |
|-----------------|--------------------|---------------------|----------------------|----------------------|
| λ <sub>1</sub>  | 58,887.44          | 52,135.03           | 74,551.41            | 71,548.36            |
| λ <sub>2</sub>  | 6,308.49           | 336.52              | 13,625.52            | 8,086.51             |
| λ <sub>3</sub>  | 112,147.38         | 57,370.42           | 120,664.29           | 122,662.60           |
| λ <sub>4</sub>  | 36.35              | 1.21                | 4.93                 | 5.90                 |
| λ <sub>5</sub>  | 115,622.65         | 36,171.24           | 117,471.82           | 123,134.82           |
| λ <sub>6</sub>  | 16,117.29          | 367.01              | 9,943.26             | 6,335.60             |
| λ <sub>7</sub>  | 37.58              | 20.54               | 99.82                | 63.80                |
| λ <sub>8</sub>  | 36,849.11          | 9,451.02            | 38,580.06            | 33,629.18            |
| λ <sub>9</sub>  | 35,978.84          | 10,410.18           | 39,407.07            | 38,776.57            |
| λ <sub>10</sub> | 1,124.24           | 36.27               | 1,513.13             | 1,297.55             |

Table 3: Average number of B&B nodes over solved instances from all datasets for each model and  $\lambda$ -vector, rounded to two decimal places.

of solved instances. Moreover, our models solve the most instances overall on  $\lambda_5$  and  $\lambda_7$ , and DOMP<sub>θB</sub> is slightly ahead on  $\lambda_8$  and  $\lambda_9$ .

Finally, Table 4 confirms a known relationship between the number of opened facilities  $p$  and overall computational difficulty. Across all four formulations, the highest average runtimes occur at  $p = \lfloor \frac{n}{10} \rfloor$ , while

| $p$                            | DOMP <sub>θB</sub> | mDOMP <sub>rB</sub> | mDOMP <sub>sB1</sub> | mDOMP <sub>sB2</sub> |
|--------------------------------|--------------------|---------------------|----------------------|----------------------|
| $\lfloor \frac{n}{10} \rfloor$ | 3166.29 (173)      | 3505.64 (206)       | 3167.49 (176)        | 3175.89 (180)        |
| $\lfloor \frac{n}{4} \rfloor$  | 3137.12 (184)      | 3284.19 (187)       | 3116.79 (179)        | 3109.57 (180)        |
| $\lfloor \frac{n}{3} \rfloor$  | 3135.53 (181)      | 3316.12 (189)       | 3153.02 (179)        | 3152.18 (180)        |
| $\lfloor \frac{n}{2} \rfloor$  | 2922.61 (167)      | 2972.81 (162)       | 2917.93 (169)        | 2920.06 (169)        |

Table 4: Average runtime in seconds and count of unsolved instances in parentheses (out of all 450 instances per  $p$ ) for different numbers of opened facilities  $p$ . Runtimes are rounded to two decimal places.

the lowest are observed at  $p = \lfloor \frac{n}{2} \rfloor$ . For  $p = \lfloor \frac{n}{4} \rfloor$  and  $p = \lfloor \frac{n}{3} \rfloor$ , the average runtimes are similar, with no clear ordering between the two values. Also, all four models have the lowest number of unsolved instances at  $p = \lfloor \frac{n}{2} \rfloor$ . This observation is somewhat unintuitive, since the number of candidate solutions is largest in this case. We do not have a sound explanation for this phenomenon at the moment, but this behavior is consistent with previous studies (Labbé et al., 2017; Marín et al., 2020; Ljubić et al., 2024).

### 4.3.2 Gaps and bounds

Next, we analyze relative optimality gaps at termination to assess solution quality in more detail, especially for non-optimal runs. Let  $\bar{z}$  denote the objective value of the incumbent solution and  $\underline{z}$  the best lower bound obtained during the optimization. Since these values may have different signs in our setting, we use the following symmetric normalization to measure relative optimality gaps:

$$\text{gap} := \frac{\bar{z} - \underline{z}}{\max\{1, |\bar{z}|, |\underline{z}|\}} \in [0, 2].$$

If no solution has been found, the gap is set to the formula’s maximum possible value of 2. Figure 4 shows, for each model, the cumulative fraction of instances (y-axis) achieving a given gap value (x-axis) on the random instances. The plot clearly indicates that  $\text{mDOMP}_{rB}$  is the least robust model on this dataset, in the sense

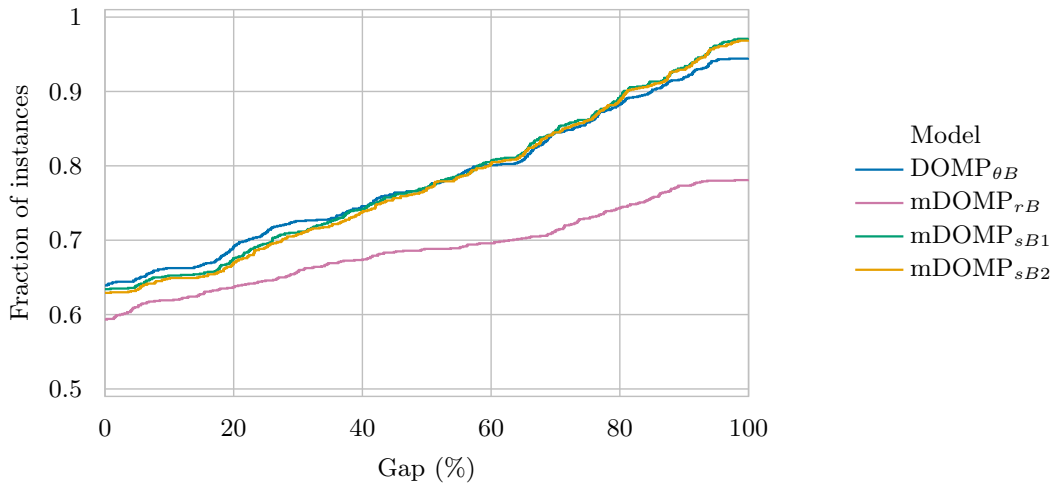


Figure 4: Cumulative distribution of optimality gaps on random instances at termination.

that it solves fewer instances than all other models within any given gap value. The other three models are more competitive and  $\text{DOMP}_{\theta B}$  holds a slight advantage for small gaps up to roughly 35%.  $\text{mDOMP}_{sB1}$  and  $\text{mDOMP}_{sB2}$  are the most successful in producing any incumbent solution (approximately 97%). The cumulative gap curves of the latter two models behave similarly, with  $\text{mDOMP}_{sB1}$  reaching slightly better gaps than  $\text{mDOMP}_{sB2}$ .

To understand the drivers behind these gaps, Table 5 reports the number of times each model achieved the best lower or upper bound on the random dataset. These numbers confirm the weakness of  $\text{mDOMP}_{rB}$ : it

|                  | $\text{DOMP}_{\theta B}$ | $\text{mDOMP}_{rB}$ | $\text{mDOMP}_{sB1}$ | $\text{mDOMP}_{sB2}$ |
|------------------|--------------------------|---------------------|----------------------|----------------------|
| Best lower bound | 927                      | 845                 | 916                  | 938                  |
| Best upper bound | 902                      | 730                 | 902                  | 881                  |

Table 5: Number of random instances where each model achieved the best lower and upper bounds within a relative tolerance of  $10^{-4}$  compared to all other models.

obtains both the least best lower bounds and upper bounds. In contrast, our proposed models and  $\text{DOMP}_{\theta B}$  consistently find higher quality bounds. An interesting observation is that  $\text{mDOMP}_{sB1}$  finds better upper bounds than  $\text{mDOMP}_{sB2}$ , but weaker lower bounds, largely equalizing the overall gap performance.

Next, we turn to the Beasley dataset. Figure 5 shows the cumulative gap distribution for these structured instances. In contrast to the random instances,  $\text{mDOMP}_{rB}$  dominates the gap comparison in this case.

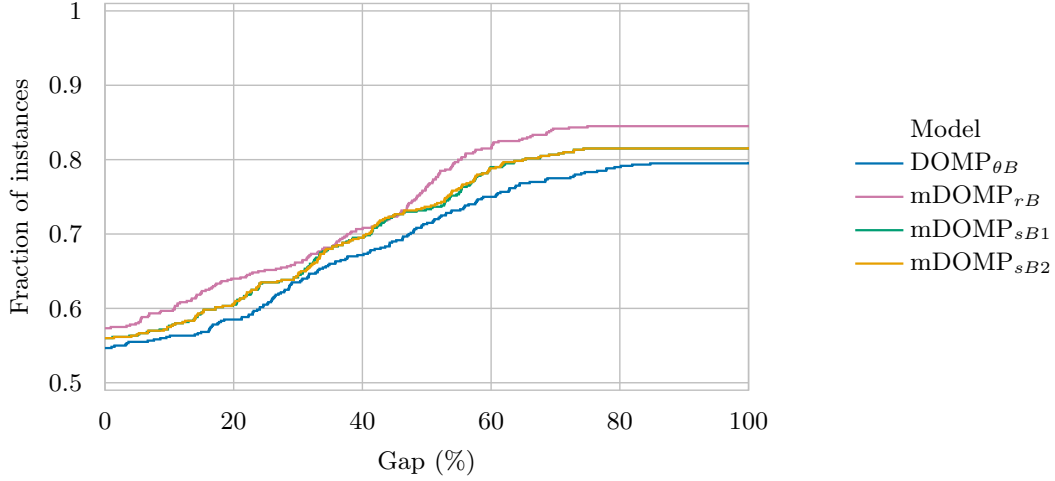


Figure 5: Cumulative distribution of optimality gaps on Beasley instances at termination.

It achieves gaps below 20% in roughly 65% of the instances and flattens out at an 85% incumbent rate. Conversely,  $\text{DOMP}_{\theta B}$  is the weakest model here. Our formulations,  $\text{mDOMP}_{sB1}$  and  $\text{mDOMP}_{sB2}$ , sit in the middle, demonstrating robustness across both data types. Table 6 confirms the reversed gap performance pattern:  $\text{mDOMP}_{rB}$  benefits from the repeated cost structure of the Beasley dataset to find the best lower bound in 482 instances and the best upper bound in 434, significantly outperforming all other formulations.

|                  | $\text{DOMP}_{\theta B}$ | $\text{mDOMP}_{rB}$ | $\text{mDOMP}_{sB1}$ | $\text{mDOMP}_{sB2}$ |
|------------------|--------------------------|---------------------|----------------------|----------------------|
| Best lower bound | 361                      | 482                 | 377                  | 390                  |
| Best upper bound | 368                      | 434                 | 379                  | 377                  |

Table 6: Number of Beasley instances where each model achieved the best lower and upper bounds within a relative tolerance of  $10^{-4}$  compared to all other models.

Finally, Table 7 details the average optimality gaps grouped by the number of opened facilities  $p$ . Consis-

| $p$                            | $\text{DOMP}_{\theta B}$ | $\text{mDOMP}_{rB}$ | $\text{mDOMP}_{sB1}$ | $\text{mDOMP}_{sB2}$ |
|--------------------------------|--------------------------|---------------------|----------------------|----------------------|
| $\lfloor \frac{n}{10} \rfloor$ | 38.92%                   | 51.04%              | 41.23%               | 42.46%               |
| $\lfloor \frac{n}{4} \rfloor$  | 34.77%                   | 48.34%              | 30.55%               | 30.92%               |
| $\lfloor \frac{n}{3} \rfloor$  | 34.41%                   | 48.97%              | 29.15%               | 29.12%               |
| $\lfloor \frac{n}{2} \rfloor$  | 32.84%                   | 47.98%              | 27.07%               | 27.08%               |

Table 7: Average gaps over all instances from both datasets for different numbers of opened facilities  $p$ , rounded to two decimal places.

tent with the runtime and timeout metrics discussed in Section 4.3.1, instances with  $p = \lfloor \frac{n}{10} \rfloor$  yield the largest average gaps across all formulations. As  $p$  increases to  $\lfloor \frac{n}{2} \rfloor$ , the average gaps are significantly reduced and there are no big differences between  $p = \lfloor \frac{n}{3} \rfloor$  and  $p = \lfloor \frac{n}{4} \rfloor$  for each model. Notably, our models  $\text{mDOMP}_{sB1}$  and  $\text{mDOMP}_{sB2}$  provide the tightest average gaps for medium to large  $p$  values ( $p \geq \lfloor \frac{n}{4} \rfloor$ ), outperforming both reference models.

### 4.3.3 Separation procedures

Finally, we have a closer look at the efficiency of the separation procedures. Particularly, we report in Figure 6, for each model, the average time spent in callbacks and the average number of added lazy constraints per random instance. Since  $\text{mDOMP}_{rB}$  adds by far the largest number of lazy constraints, it also spends, on

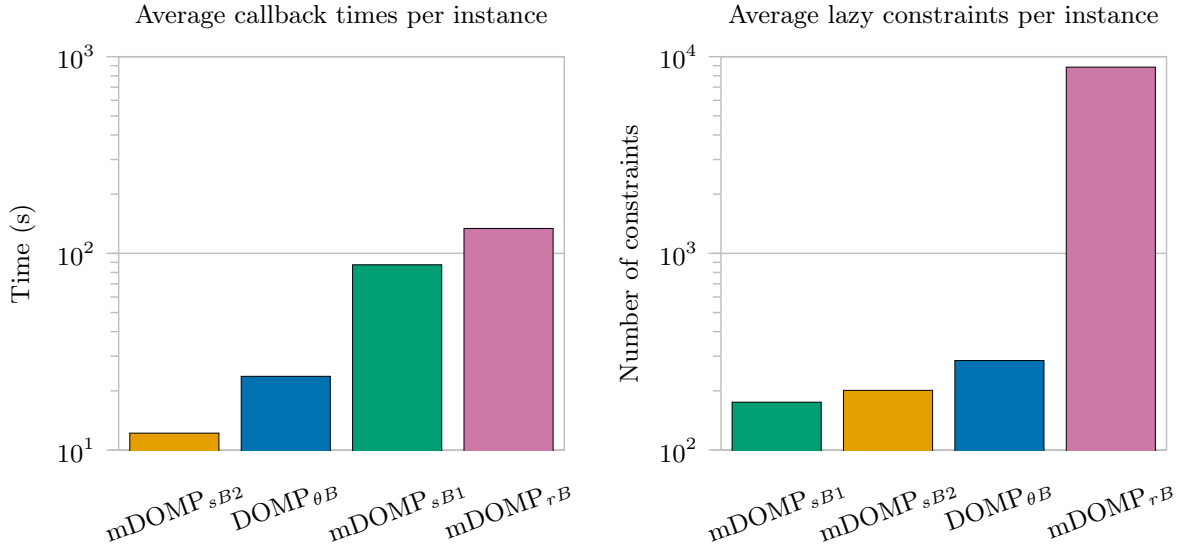


Figure 6: Average callback times and numbers of added lazy constraints across all random instances.

average, more time in the callback than the other models.  $\text{DOMP}_{\theta B}$  generates roughly 1.5 times as many lazy constraints as  $\text{mDOMP}_{sB1}$  and  $\text{mDOMP}_{sB2}$ , yet its average callback time lies between the two, lower than for  $\text{mDOMP}_{sB1}$  but higher than for  $\text{mDOMP}_{sB2}$ . Notably,  $\text{mDOMP}_{sB1}$  and  $\text{mDOMP}_{sB2}$  achieve gaps and bounds competitive with the benchmark model  $\text{DOMP}_{\theta B}$  on the random instances (see Figure 4 and Table 5) while adding fewer lazy constraints, indicating that their cuts are rather strong and effective. Further, the large number of comparatively weaker cuts generated by  $\text{mDOMP}_{rB}$  contributes to its inferior performance on this dataset.

We further comment on the observed differences in callback times between  $\text{mDOMP}_{sB1}$  and  $\text{mDOMP}_{sB2}$ , which stem from the distinct implementations of Algorithm 4. For  $\text{mDOMP}_{sB1}$ , the computational bottleneck is the evaluation of (15) to determine the  $v^f$ -variables. This requires a for-loop over all  $h \in [g]$  with an inner loop over  $k \in [n]$  while  $f(k) < h$ . In contrast,  $\text{mDOMP}_{sB2}$  computes the  $u^f$ -values via (20), using a for-loop over  $l \in [n]$  and an inner loop over  $k \in [g-1]_0$  while  $f(k) < l$ . In the worst case, both implementations perform  $ng$  inner iterations. As the cost matrices are randomly drawn from a sufficiently large set of values, we typically have  $g \gg n$ . In  $\text{mDOMP}_{sB1}$ , the outer loop iterates over all  $g$  indices independently of the data, leading to consistently high iteration counts. On the other hand,  $\text{mDOMP}_{sB2}$  places the larger index set inside the while loop, where the termination criterion usually truncates the iterations early. Consequently, the effective number of iterations is substantially smaller for  $\text{mDOMP}_{sB2}$ , explaining its faster average callback times. Note that under different structural conditions, in particular when  $g < n$ , this effect may reverse.

On the Beasley instances, where  $g \gg n$ , the differences between  $\text{mDOMP}_{sB1}$  and  $\text{mDOMP}_{sB2}$  in their callback behavior are less significant as Figure 7 shows.  $\text{DOMP}_{\theta B}$  requires the highest average callback time on these structured instances, and its cuts appear less effective given its weaker performance noted in the previous subsection. While  $\text{mDOMP}_{rB}$  still generates the highest volume of lazy constraints, the repeated

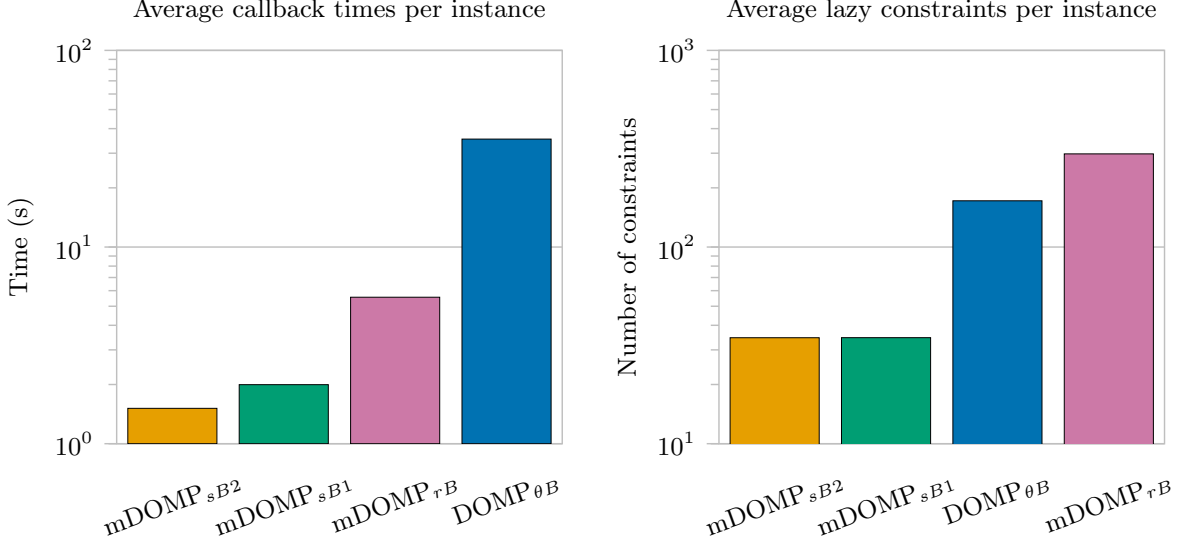


Figure 7: Average callback times and numbers of added lazy constraints across all Beasley instances.

cost entries in the Beasley matrices have the effect that the model needs fewer such constraints to sufficiently restrict the search space and arrive at good solutions, resulting in its observed dominance on this dataset.

#### 4.3.4 Summary

The computational results reveal dataset-dependent strengths of the two benchmark formulations mDOMP<sub>rB</sub> and DOMP<sub>θB</sub> and show how our proposed models perform relative to them. mDOMP<sub>rB</sub> strongly benefits from repeated entries in the cost matrix. It dominates on the Beasley dataset, but performs weakest on randomly generated instances. For DOMP<sub>θB</sub>, we observed the opposite: it excels on random data but falls behind on the Beasley instances. While mDOMP<sub>sB1</sub> and mDOMP<sub>sB2</sub> do not surpass the benchmark models on their strong dataset, they clearly outperform them on their weaker ones. This consistent performance makes our formulations a reliable, general-purpose choice across diverse cost structures and objective criteria.

Besides model comparisons, our extensive numerical study yields some insights into computational problem difficulty. In particular, all models solve roughly twice as many instances of sizes  $n \in \{100, 200\}$  from the Beasley dataset as from the random dataset, suggesting that the structured costs make these instances easier for the solver. The objective criterion is another important factor. Integer-friendly criteria like the  $p$ -median require little branching, leading to fast solve times and consistent solvability up to the largest instances with  $n = 300$  for all models. In contrast, obnoxious objectives ( $\lambda_1, \lambda_3, \lambda_5$ ), alongside negative reverse ( $\lambda_8$ ) and random non-increasing ( $\lambda_9$ ) weights, lack this integer-friendliness, resulting in large B&B trees and frequent timeouts. Also, the number of opened facilities  $p$  influences the computational difficulty: all models struggled, on average, most with  $p = \lfloor \frac{n}{10} \rfloor$  and least with  $p = \lfloor \frac{n}{2} \rfloor$  in our numerical experiments.

Finally, when interpreting the absolute runtimes and gaps reported in this study, it is important to recall that our experimental design intends to isolate the theoretical strength of the formulations. Therefore, we disabled Gurobi’s presolve, heuristics, solver-generated cuts, and multi-threading. Moreover, we omitted further enhancements like tailored valid inequalities or variable fixing. Enabling solver features and model enhancements is expected to improve the practical scalability of all tested models.

## 5 Conclusion and further research

In this paper, we studied the TP under a Monge cost structure and showed how the optimal solution obtained by the northwest-corner rule can be translated into explicit solution formulas for the corresponding dual problem. Because the TP frequently arises, often as a substructure of more complex optimization models, such dual formulas can enable the application of advanced optimization techniques and provide deeper insight into the structure of these problems. To illustrate this potential, we employed the derived formulas to construct two Benders reformulations,  $\text{mDOMP}_{sB1}$  and  $\text{mDOMP}_{sB2}$ , of a MILP formulation for the DOMP with a non-increasing objective weight vector.

Both formulations demonstrate strong computational performance and prove to be competitive with two state-of-the-art benchmark models from the literature. A particular strength lies in their robustness across a diverse range of instances with different cost structures and objective criteria. While the two benchmark models  $\text{DOMP}_{\theta B}$  and  $\text{mDOMP}_{rB}$  have clear data-specific strengths and weaknesses, our formulations' good performance is less dependent on a specific instance type. Although our new models describe the same polytope, the numerical experiments reveal that  $\text{mDOMP}_{sB1}$  spends, on average, significantly more time in the separation callbacks than  $\text{mDOMP}_{sB2}$  on instances with a large number of distinct cost entries. This highlights that one of the two models may be preferred over the other in practice. Furthermore, our experiments empirically verify the computational difficulty of obnoxious location criteria, contrasting them with classical and integer-friendly objectives like the  $p$ -median problem.

A promising direction for future research is the application of the derived formulas to other problems, some of which were mentioned in the introduction, where the TP under Monge costs arises naturally. Another avenue is the investigation of a possible generalization of the optimal dual formulas to the multidimensional TP with Monge costs, for which an optimal greedy algorithm is also known (cf. Burkard et al., 1996, Theorem 6.5). Finally, ongoing research aims to extend the models  $\text{mDOMP}_{sB1}$  and  $\text{mDOMP}_{sB2}$ , currently restricted to non-increasing objective vectors  $\lambda$ , to arbitrary  $\lambda$ .

## Acknowledgements

This work was supported by the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) [project number 541767835] and the Agencia Estatal de Investigación (AEI, MICIU/AEI/10.13039/501100011033) [project numbers PCI2024-155024-2, PID2020-114594GB-C21, PID2024-156594NB-C21, CEX2024-001517-M].

The numerical experiments were performed on the computational resource bwUniCluster funded by the Ministry of Science, Research and the Arts Baden-Württemberg and the Universities of the State of Baden-Württemberg, Germany, within the framework program bwHPC.

## Declaration of interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper. The funding agencies had no role in the design of the study; in the collection, analysis, or interpretation of data; in the writing of the manuscript; or in the decision to submit the article for publication.

# Declaration of generative AI in the manuscript preparation process

During the preparation of this work, the authors used ChatGPT (OpenAI) and Gemini (Google) to improve the readability and language of the manuscript and to assist in drafting selected code snippets. All AI-generated text and code were carefully reviewed, validated, and, where necessary, revised and adapted by the authors. The authors take full responsibility for the content of the published article.

## References

- Aggarwal, A., Barnoy, A., Khuller, S., Kravets, D., & Schieber, B. (1995). Efficient Minimum Cost Matching and Transportation Using the Quadrangle Inequality. *Journal of Algorithms*, 19(1), 116–143. <https://doi.org/10.1006/jagm.1995.1030>
- Bazaraa, M. S., Jarvis, J. J., & Sherali, H. D. (2010). *Linear Programming and Network Flows* (4th ed.). John Wiley & Sons.
- Beasley, J. E. (1985). A note on solving large  $p$ -median problems. *European Journal of Operational Research*, 21(2), 270–273. [https://doi.org/10.1016/0377-2217\(85\)90040-2](https://doi.org/10.1016/0377-2217(85)90040-2)
- Bigler, T. (2024). A matheuristic for locating obnoxious facilities. *Computers & Operations Research*, 166, Article 106602. <https://doi.org/10.1016/j.cor.2024.106602>
- Boland, N., Domínguez-Marín, P., Nickel, S., & Puerto, J. (2006). Exact procedures for solving the discrete ordered median problem. *Computers & Operations Research*, 33(11), 3270–3300. <https://doi.org/10.1016/j.cor.2005.03.025>
- Burkard, R. E. (2007). Monge properties, discrete convexity and applications. *European Journal of Operational Research*, 176(1), 1–14. <https://doi.org/10.1016/j.ejor.2005.04.050>
- Burkard, R. E., Klinz, B., & Rudolf, R. (1996). Perspectives of Monge properties in optimization. *Discrete Applied Mathematics*, 70(2), 95–161. [https://doi.org/10.1016/0166-218X\(95\)00103-X](https://doi.org/10.1016/0166-218X(95)00103-X)
- Çela, E., Deineko, V., & Woeginger, G. J. (2018). New special cases of the Quadratic Assignment Problem with diagonally structured coefficient matrices. *European Journal of Operational Research*, 267(3), 818–834. <https://doi.org/10.1016/j.ejor.2017.12.024>
- Cherkesly, M., Contardo, C., & Gruson, M. (2025). Ranking Decomposition for the Discrete Ordered Median Problem. *INFORMS Journal on Computing*, 37(2), 230–248. <https://doi.org/10.1287/ijoc.2023.0059>
- Chiang, Y.-I. & Lin, C.-C. (2017). Compact Model for the Obnoxious  $p$ -Median Problem. *American Journal of Operations Research*, 7(6), 348–355. <https://doi.org/10.4236/ajor.2017.76026>
- Church, R. L. & Drezner, Z. (2022). Review of obnoxious facilities location problems. *Computers & Operations Research*, 138, Article 105468. <https://doi.org/10.1016/j.cor.2021.105468>
- Colmenar, J. M., Greistorfer, P., Martí, R., & Duarte, A. (2016). Advanced Greedy Randomized Adaptive Search Procedure for the Obnoxious  $p$ -Median problem. *European Journal of Operational Research*, 252(2), 432–442. <https://doi.org/10.1016/j.ejor.2016.01.047>

- Dahl, G. (2008). Transportation matrices with staircase patterns and majorization. *Linear Algebra and its Applications*, 429(7), 1840–1850. <https://doi.org/10.1016/j.laa.2008.05.019>
- Dantzig, G. B. (1951). Application of the Simplex Method to a Transportation Problem. *Activity Analysis of Production and Allocation*, 359–373. John Wiley & Sons. Cowles Commission Monograph No. 13.
- Deleplanque, S., Labbé, M., Ponce, D., & Puerto, J. (2020). A Branch-Price-and-Cut Procedure for the Discrete Ordered Median Problem. *INFORMS Journal on Computing*, 32(3), 582–599. <https://doi.org/10.1287/ijoc.2019.0915>
- Domínguez, E. & Marín, A. (2020). Discrete ordered median problem with induced order. *TOP*, 28(3), 793–813. <https://doi.org/10.1007/s11750-020-00570-1>
- Domínguez-Marín, P. (2003). *The Discrete Ordered Median Problem: Models and Solution Methods*. Combinatorial Optimization. Springer New York. <https://doi.org/10.1007/978-1-4419-8511-8>
- Domínguez-Marín, P., Nickel, S., Hansen, P., & Mladenović, N. (2005). Heuristic Procedures for Solving the Discrete Ordered Median Problem. *Annals of Operations Research*, 136(1), 145–173. <https://doi.org/10.1007/s10479-005-2043-3>
- Espejo, I., Marín, A., Muñoz-Ocaña, J. M., & Rodríguez-Chía, A. M. (2023). A new formulation and branch-and-cut method for single-allocation hub location problems. *Computers & Operations Research*, 155, Article 106241. <https://doi.org/10.1016/j.cor.2023.106241>
- Fischetti, M., Ljubić, I., & Sinnl, M. (2016). Benders decomposition without separability: A computational study for capacitated facility location problems. *European Journal of Operational Research*, 253(3), 557–569. <https://doi.org/10.1016/j.ejor.2016.03.002>
- Ford, L. R. & Fulkerson, D. R. (1956). Solving the Transportation Problem. *Management Science*, 3(1), 24–32. <https://doi.org/10.1287/mnsc.3.1.24>
- Gurobi Optimization, LLC (2026). *Gurobi Optimizer Reference Manual*. Retrieved from <https://www.gurobi.com>. Accessed January 11, 2026.
- Hardy, G. H., Littlewood, J. E., & Pólya, G. (1934). *Inequalities*. Cambridge University Press.
- Herrán, A., Colmenar, J. M., Martí, R., & Duarte, A. (2020). A parallel variable neighborhood search approach for the obnoxious  $p$ -median problem. *International Transactions in Operational Research*, 27(1), 336–360. <https://doi.org/10.1111/itor.12510>
- Hillbrecht, S. (2025). Bilevel Optimization of the Kantorovich Problem and its Quadratic Regularization Part III: The Finite-Dimensional Case. *arXiv preprint*. <https://doi.org/10.48550/arXiv.2406.08992>
- Hitchcock, F. L. (1941). The Distribution of a Product from Several Sources to Numerous Localities. *Journal of Mathematics and Physics*, 20(1-4), 224–230. <https://doi.org/10.1002/sapm1941201224>
- Hoffman, A. J. (1963). On Simple Linear Programming Problems. *Proceedings of Symposia in Pure Mathematics*, volume 7, 317–327. American Mathematical Society.
- Kacher, Y. & Singh, P. (2021). A Comprehensive Literature Review on Transportation Problems. *International Journal of Applied and Computational Mathematics*, 7(5), Article 206. <https://doi.org/10.1007/s40819-021-01134-y>

- Kalczynski, P. & Drezner, Z. (2026). The obnoxious facilities  $p$ -center problem with forbidden regions. *Computers & Operations Research*, 185, Article 107297. <https://doi.org/10.1016/j.cor.2025.107297>
- Kantorovich, L. V. (1942). On the translocation of masses. *Doklady Akademii Nauk SSSR*, 37(7–8), 199–201.
- Koopmans, T. C. (1949). Optimum Utilization of the Transportation System. *Econometrica*, 17, 136–146. <https://doi.org/10.2307/1907301>
- Labbé, M., Ponce, D., & Puerto, J. (2017). A comparative study of formulations and solution methods for the discrete ordered  $p$ -median problem. *Computers & Operations Research*, 78, 230–242. <https://doi.org/10.1016/j.cor.2016.06.004>
- Ljubić, I., Pozo, M. A., Puerto, J., & Torrejon, A. (2024). Benders decomposition for the discrete ordered median problem. *European Journal of Operational Research*, 317(3), 858–874. <https://doi.org/10.1016/j.ejor.2024.04.030>
- Marín, A., Nickel, S., Puerto, J., & Velten, S. (2009). A flexible model and efficient solution strategies for discrete location problems. *Discrete Applied Mathematics*, 157(5), 1128–1145. <https://doi.org/10.1016/j.dam.2008.03.013>
- Marín, A., Ponce, D., & Puerto, J. (2020). A fresh view on the Discrete Ordered Median Problem based on partial monotonicity. *European Journal of Operational Research*, 286(3), 839–848. <https://doi.org/10.1016/j.ejor.2020.04.023>
- Monge, G. (1781). *Mémoire sur la théorie des déblais et des remblais [Memoir on the theory of cut and fill]*. Imprimerie royale.
- Nickel, S. (2001). Discrete Ordered Weber Problems. *Operations Research Proceedings*, 71–76. Springer Berlin, Heidelberg. [https://doi.org/10.1007/978-3-642-56656-1\\_12](https://doi.org/10.1007/978-3-642-56656-1_12)
- Nickel, S. & Puerto, J. (2005). *Location Theory*. Springer Berlin, Heidelberg. <https://doi.org/10.1007/3-540-27640-8>
- Nickel, S., Puerto, J., Ramoser, S., & Torrejon, A. (2026). *Instance and solution data for 'Revisiting transportation problems under Monge costs with applications to the Discrete Ordered Median Problem'*. Mendeley Data, V2. <https://doi.org/10.17632/kk8ms9rsby.2>
- Nickel, S. & Velten, S. (2017). Optimization problems with flexible objectives: A general modeling approach and applications. *European Journal of Operational Research*, 258(1), 79–88. <https://doi.org/10.1016/j.ejor.2016.08.045>
- Pozo, M. A., Puerto, J., & Rodríguez-Chía, A. M. (2021). The ordered median tree of hubs location problem. *TOP*, 29(1), 78–105. <https://doi.org/10.1007/s11750-020-00572-z>
- Pozo, M. A., Puerto, J., & Torrejon, A. (2024). The Ordered Median Tree Location Problem. *Computers & Operations Research*, 169, Article 106746. <https://doi.org/10.1016/j.cor.2024.106746>
- Puerto, J. (2008). A New Formulation of the Capacitated Discrete Ordered Median Problems with  $\{0, 1\}$ -Assignment. *Operations Research Proceedings 2007*, 165–170. Springer. [https://doi.org/10.1007/978-3-540-77903-2\\_26](https://doi.org/10.1007/978-3-540-77903-2_26)

- Puerto, J., Ramos, A., & Rodríguez-Chía, A. (2011). Single-allocation ordered median hub location problems. *Computers & Operations Research*, 38(2), 559–570. <https://doi.org/10.1016/j.cor.2010.07.018>
- Puerto, J., Rodríguez-Chía, A. M., & Tamir, A. (2017). Revisiting  $k$ -sum optimization. *Mathematical Programming*, 165(2), 579–604. <https://doi.org/10.1007/s10107-016-1096-1>
- Puerto, J. & Tamir, A. (2005). Locating tree-shaped facilities using the ordered median objective. *Mathematical Programming*, 102(2), 313–338. <https://doi.org/10.1007/s10107-004-0547-2>
- Queyranne, M., Spieksma, F., & Tardella, F. (1998). A General Class of Greedily Solvable Linear Programs. *Mathematics of Operations Research*, 23(4), 892–908. <https://doi.org/10.1287/moor.23.4.892>
- ReVelle, C. (1993). Facility siting and integer-friendly programming. *European Journal of Operational Research*, 65(2), 147–158. [https://doi.org/10.1016/0377-2217\(93\)90329-L](https://doi.org/10.1016/0377-2217(93)90329-L)
- Rubner, Y., Tomasi, C., & Guibas, L. (1998). A Metric for Distributions with Applications to Image Databases. *Sixth International Conference on Computer Vision (IEEE Cat. No.98CH36271)*, 59–66. <https://doi.org/10.1109/ICCV.1998.710701>
- Singh, S., Khandelwal, A., & Puri, M. C. (2008). Bilevel time minimizing transportation problem. *Discrete Optimization*, 5(4), 714–723. <https://doi.org/10.1016/j.disopt.2008.04.004>
- Stanley, R. P. (1997). *Enumerative Combinatorics: Volume 1* (2nd ed.). Cambridge University Press.
- Tseytlin, Y. & Heese, H. S. (2019). Allocation under a general substitution structure. *European Journal of Operational Research*, 277(2), 492–506. <https://doi.org/10.1016/j.ejor.2019.02.049>
- Tseytlin, Y. & Heese, H. S. (2023). Inventory allocation with full downward substitution and monotone cost differences. *European Journal of Operational Research*, 307(1), 130–139. <https://doi.org/10.1016/j.ejor.2022.08.027>
- Werman, M., Peleg, S., & Rosenfeld, A. (1985). A distance metric for multidimensional histograms. *Computer Vision, Graphics, and Image Processing*, 32(3), 328–336. [https://doi.org/10.1016/0734-189X\(85\)90055-6](https://doi.org/10.1016/0734-189X(85)90055-6)
- Yemelichev, V. A., Kovalev, M. M., & Kravtsov, M. K. (1984). *Polytopes, Graphs and Optimisation*. Cambridge University Press.
- Zhang, N. & Xie, F. (2025). A Review of Research on the Transportation Problem. *Open Journal of Applied Sciences*, 15(5), 1168–1177. <https://doi.org/10.4236/ojapps.2025.155081>